

+

Informatik 2 – Grafikprogrammierung

K

A



©image: various

Echtzeit Computer Grafik

Sprites



Polygone



Raytracing
(Real-Time)



Commander Keen 1 (1990)

Super Mario Kart (1992)

Quake (1996)

Battlefield V (2018)

t

Erstes Side-Scrolling Spiel auf PC
(Adaptive-Tile-Refresh-Technik)

Mode 7

erster Ego-Shooter mit Gegner
und Gegenstände aus
Polygonen

mit RT-Raytracing



Wer einen Pixel auf dem Bildschirm setzen kann,
hat alle notwendigen Werkzeuge zur Grafikprogrammierung.

Pixel mit windows.h



```
#include <iostream>
#include <windows.h>

int main()
{
    std::cout << "Es war ein Pixel\n";
    HWND myconsole = GetConsoleWindow();
    HDC mydc = GetDC(myconsole);
    COLORREF COLOR = RGB(255, 0, 0);
    SetPixelV(mydc, 1, 1, COLOR) == 0);

    return 0;
}
```

Microsoft Visual Studio-Debugging-Konsole

Es war ein Pixel

C:\EigeneLokaleDaten\CPP\Grafik_Programmieren_VL\x64\Debug\Grafik_Programmieren_VL.e
0" beendet.

Drücken Sie eine beliebige Taste, um dieses Fenster zu schließen.

Pixel mit windows.h



```
#include <iostream>
#include <windows.h>

int main()
{
    std::cout << "Es war ein Pixel\n";
    HWND myconsole = GetConsoleWindow();
    HDC mydc = GetDC(myconsole);
    COLORREF COLOR = RGB(255, 0, 0);
    SetPixelV(mydc, 1, 1, COLOR) == 0;

    return 0;
}
```



Microsoft Visual Studio-Debugging-Konsole

Es war ein Pixel

C:\EigeneLokaleDaten\CPP\Grafik\Pixel\main.cpp:10:1: error: 'SetPixelV' does not exist; did you mean 'SetPixel'?
0" beendet.

Drücken Sie eine beliebige Taste

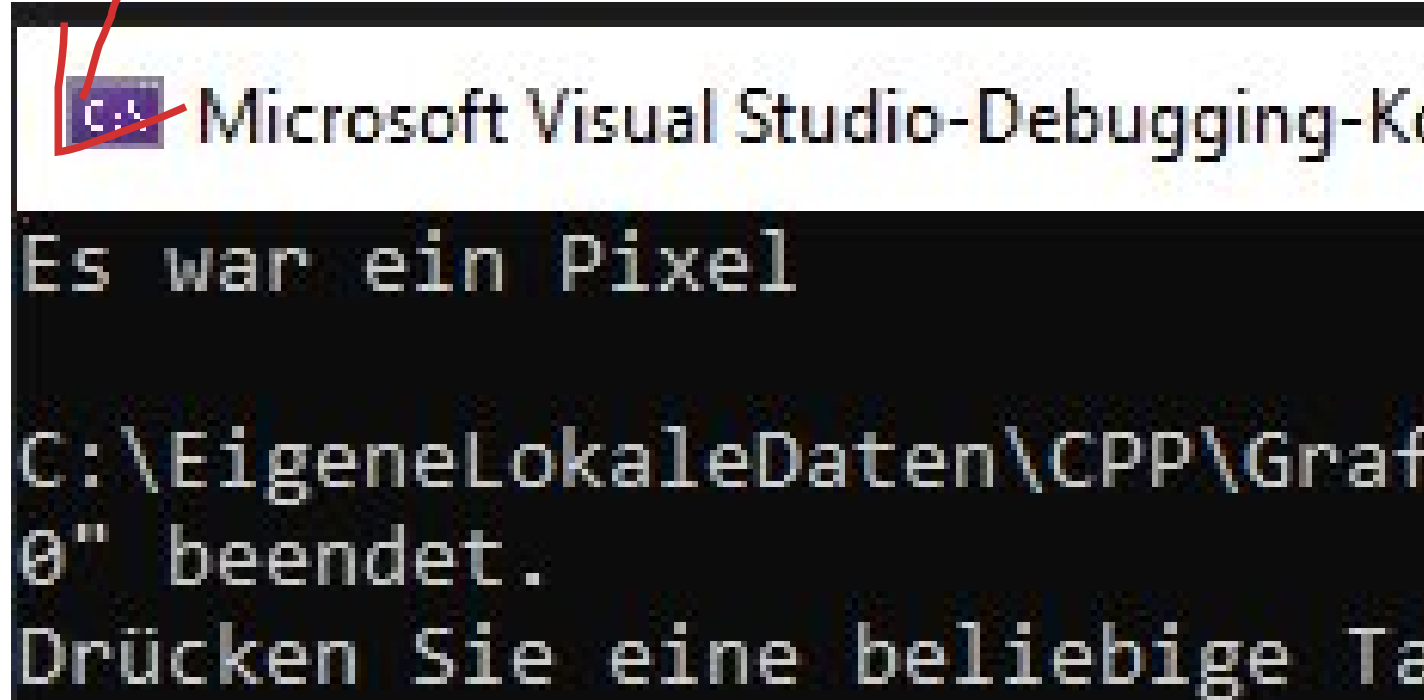
Pixel mit windows.h



```
#include <iostream>
#include <windows.h>

int main()
{
    std::cout << "Es war ein Pixel\n";
    HWND myconsole = GetConsoleWindow();
    HDC mydc = GetDC(myconsole);
    COLORREF COLOR = RGB(255, 0, 0);
    SetPixelV(mydc, 1, 1, COLOR) == 0;

    return 0;
}
```



Microsoft Visual Studio-Debugging-K

Es war ein Pixel

C:\EigeneLokaleDaten\CPP\Graf

0" beendet.

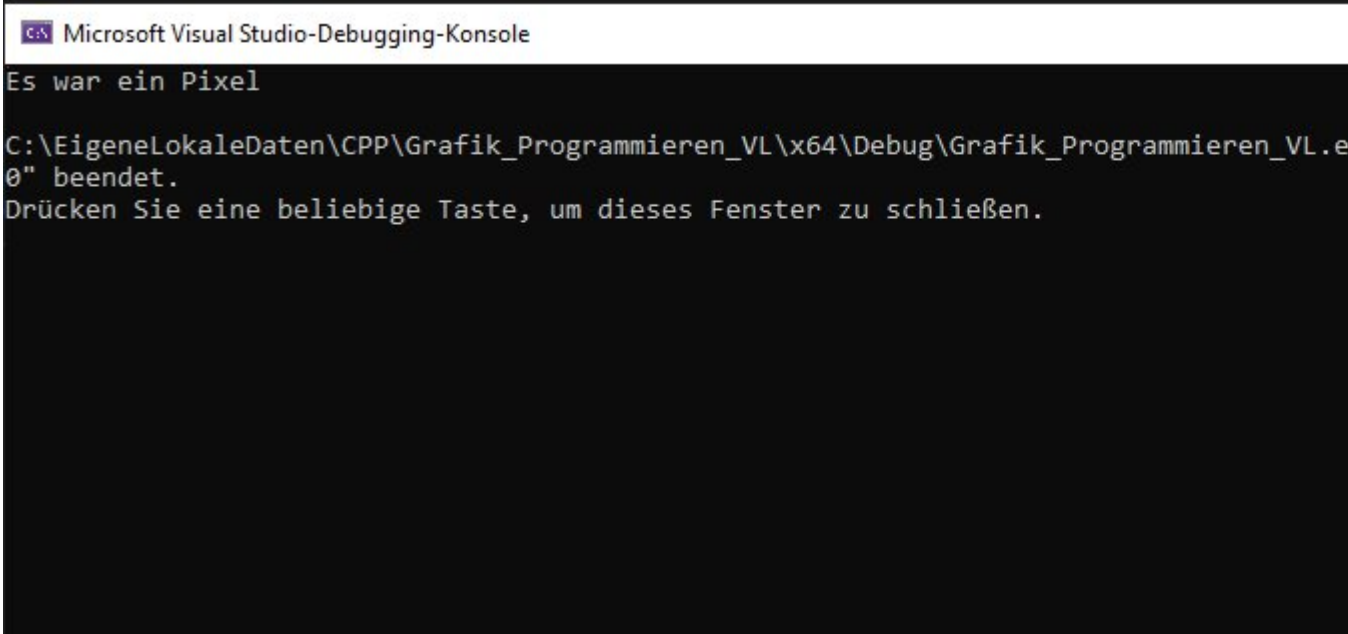
Drücken Sie eine beliebige Ta

Pixel mit windows.h



```
#include <iostream>
#include <windows.h>

int main()
{
    std::cout << "Es war ein Pixel\n";
    HWND myconsole = GetConsoleWindow();
    HDC mydc = GetDC(myconsole);
    COLORREF COLOR = RGB(255, 0, 0);
    if (SetPixelV(mydc, 1, 1, COLOR) == 0)
        std::cout << "ERROR\n";
    return 0;
}
```



```
Microsoft Visual Studio-Debugging-Konsole

Es war ein Pixel

C:\EigeneLokaleDaten\CPP\Grafik_Programmieren_VL\x64\Debug\Grafik_Programmieren_VL.e
0" beendet.
Drücken Sie eine beliebige Taste, um dieses Fenster zu schließen.
```

- Leider nicht bei allen Windows/Konsolen Versionen/Typen möglich -> Fehler abfangen.

Pixel mit windows.h



```
#include <iostream>
#include <windows.h>

int main()
{
    std::cout << "Es war ein Pixel\n";
    HWND myconsole = GetConsoleWindow();
    HDC mydc = GetDC(myconsole);
    COLORREF COLOR = RGB(255, 0, 0);
    if (SetPixelV(mydc, 1, 1, COLOR) == 0)
        std::cout << "ERROR\n";
    return 0;
}
```

```
Microsoft Visual Studio-Debugging-Konsole

Es war ein Pixel

C:\EigeneLokaleDaten\CPP\Grafik_Programmieren_VL\x64\Debug\Grafik_Programmieren_VL.e
0" beendet.
Drücken Sie eine beliebige Taste, um dieses Fenster zu schließen.
```

- Leider nicht bei allen Windows/Konsolen Versionen/Typen möglich -> Fehler abfangen.
- Daher SDL Programmieren (plattformunabhängig)

Pixeleffekte



<https://youtu.be/6hE5sEh0pwl>

Pixelexeffekte



```
#include <iostream>
#include <windows.h>

COLORREF COLOR_Arr[] = {
    RGB(0x0, 0x0, 0x0),
    //RGB(0x07, 0x07, 0x07),
    RGB(0x1F, 0x07, 0x07),
    RGB(0x2F, 0x0F, 0x07),
    RGB(0x47, 0x0F, 0x07),
    RGB(0x57, 0x17, 0x07),
    RGB(0x67, 0x1F, 0x07),
    RGB(0x77, 0x1F, 0x07),

[.]

    for (int t = 0; t < 10000; t++) {
        for (int col = 0; col < MAXCOL; col++) {
            for (int row = 0; row < MAXROW - 1; row++) {
                firebuffer[row][col] = firebuffer[row+1][col] - rand() % 2;
                if (firebuffer[row][col] < 0) { firebuffer[row][col] = 0; }
            }
        }
    }
```

C:\EigeneLokaleDaten\CPP\BitMapsConsole\x64\Debug\BitMapsConsole.exe

Video

Pixel effekte



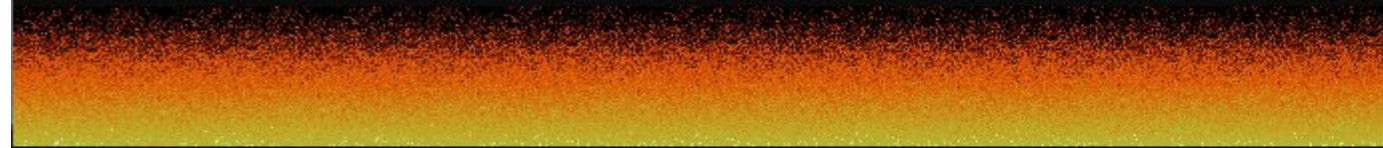
```
#include <iostream>
#include <windows.h>

COLORREF COLOR_Arr[] = {
    RGB(0x0, 0x0, 0x0),
    //RGB(0x07, 0x07, 0x07),
    RGB(0x1F, 0x07, 0x07),
    RGB(0x2F, 0x0F, 0x07),
    RGB(0x47, 0x0F, 0x07),
    RGB(0x57, 0x17, 0x07),
    RGB(0x67, 0x1F, 0x07),
    RGB(0x77, 0x1F, 0x07),
    [...]

    for (int t = 0; t < 10000; t++) {
        for (int col = 0; col < MAXCOL; col++) {
            for (int row = 0; row < MAXROW - 1; row++) {
                firebuffer[row][col] = firebuffer[row+1][col] - rand() % 2;
                if (firebuffer[row][col] < 0) { firebuffer[row][col] = 0; }
            }
        }
    }
```

C:\EigeneLokaleDaten\CPP\BitMapsConsole\x64\Debug\BitMapsConsole.exe

Hello World!
size of ColorRef: 4

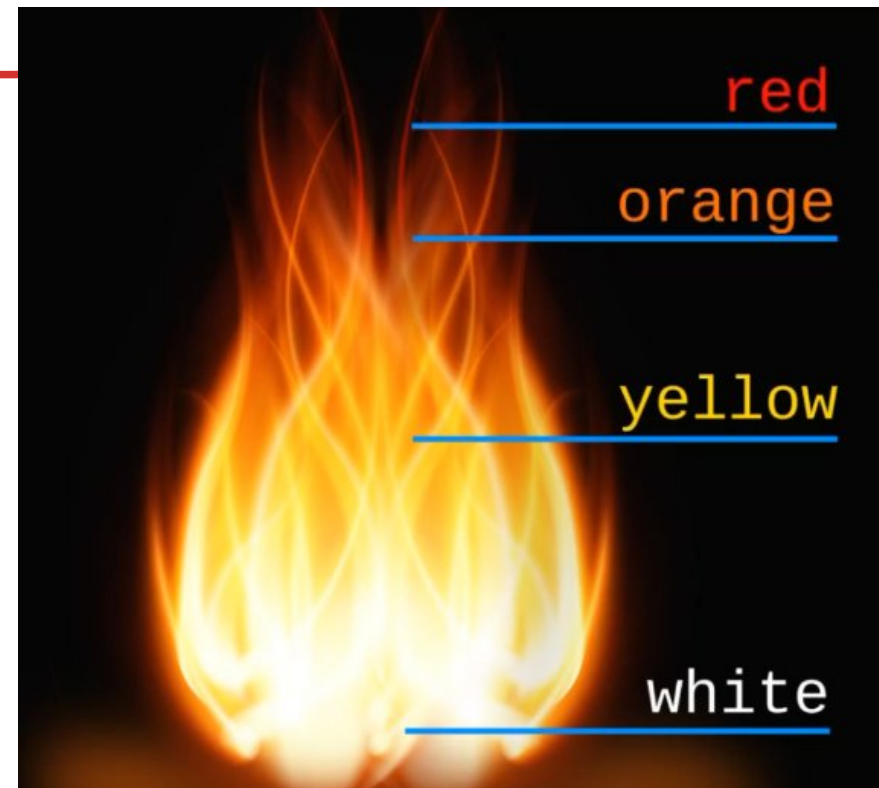


Pixelexeffekte



```
#include <iostream>
#include <windows.h>

COLORREF COLOR_Arr[] = {
    RGB(0x0, 0x0, 0x0),
    //RGB(0x07, 0x07, 0x07),
    RGB(0x1F, 0x07, 0x07),
    RGB(0x2F, 0x0F, 0x07),
    RGB(0x47, 0x0F, 0x07),
    RGB(0x57, 0x17, 0x07),
    RGB(0x67, 0x1F, 0x07),
    RGB(0x77, 0x1F, 0x07),
    RGB(0x87, 0x2F, 0x07),
    RGB(0x97, 0x3F, 0x07),
    RGB(0xA7, 0x4F, 0x07),
    RGB(0xB7, 0x5F, 0x07),
    RGB(0xC7, 0x6F, 0x07),
    RGB(0xD7, 0x7F, 0x07),
    RGB(0xE7, 0x8F, 0x07),
    RGB(0xF7, 0x9F, 0x07),
    RGB(0xFF, 0xFF, 0xFF),
};
```



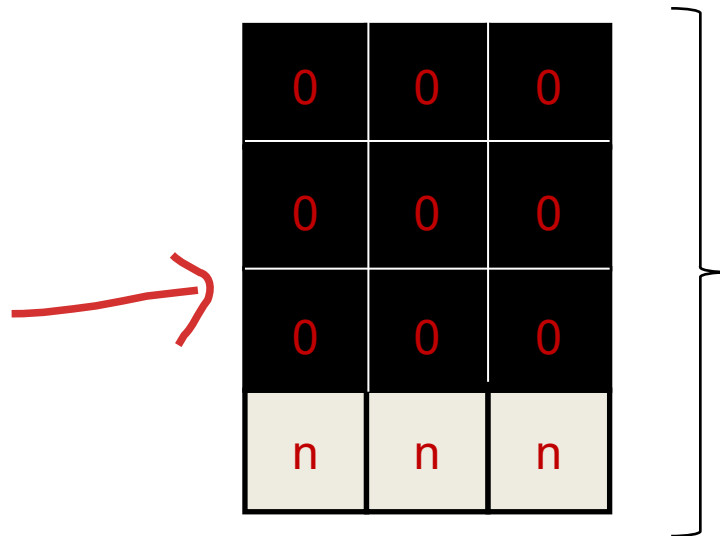
Pixel effekte



```
#include <iostream>
#include <windows.h>

COLORREF COLOR_Arr[] = {
    RGB(0x0, 0x0, 0x0),
    //RGB(0x07, 0x07, 0x07),
    RGB(0x1F, 0x07, 0x07),
    RGB(0x2F, 0x0F, 0x07),
    RGB(0x47, 0x0F, 0x07),
    RGB(0x57, 0x17, 0x07),
    RGB(0x67, 0x1F, 0x07),
    RGB(0x77, 0x1F, 0x07),
    [...]
}
```

firebuffer[row][col]



Array initialisiert:
unterste Zeile mit n

Pixelexeffekte



```
#include <iostream>
#include <windows.h>

COLORREF COLOR_Arr[] = {
    RGB(0x0, 0x0, 0x0),
    //RGB(0x07, 0x07, 0x07),
    RGB(0x1F, 0x07, 0x07),
    RGB(0x2F, 0x0F, 0x07),
    RGB(0x47, 0x0F, 0x07),
    RGB(0x57, 0x17, 0x07),
    RGB(0x67, 0x1F, 0x07),
    RGB(0x77, 0x1F, 0x07),
```

[..]

```
for (int col = 0; col < MAXCOL; col++) {
    for (int row = 0; row < MAXROW - 1; row++) {
        firebuffer[row][col] = firebuffer[row+1][col] - rand() % 2;
        if (firebuffer[row][col] < 0) { firebuffer[row][col] = 0; }
    }
}
```



0	0	0
0	0	0
n-1	n	n
n	n	n

update Farbwert basierent auf Farbwert der darunterliegenden Zeile...

Pixelexeffekte



```
#include <iostream>
#include <windows.h>

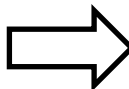
COLORREF COLOR_Arr[] = {
    RGB(0x0, 0x0, 0x0),
    //RGB(0x07, 0x07, 0x07),
    RGB(0x1F, 0x07, 0x07),
    RGB(0x2F, 0x0F, 0x07),
    RGB(0x47, 0x0F, 0x07),
    RGB(0x57, 0x17, 0x07),
    RGB(0x67, 0x1F, 0x07),
    RGB(0x77, 0x1F, 0x07),
    RGB(0x87, 0x1F, 0x07),
    RGB(0x97, 0x1F, 0x07),
    RGB(0xA7, 0x1F, 0x07),
    RGB(0xB7, 0x1F, 0x07),
    RGB(0xC7, 0x1F, 0x07),
    RGB(0xD7, 0x1F, 0x07),
    RGB(0xE7, 0x1F, 0x07),
    RGB(0xF7, 0x1F, 0x07),
    RGB(0xFF, 0x00, 0x00),
    RGB(0xFF, 0x66, 0x00),
    RGB(0xFF, 0xCC, 0x00),
    RGB(0xFF, 0xFF, 0x00),
    RGB(0xFF, 0xFF, 0x66),
    RGB(0xFF, 0xFF, 0xCC),
    RGB(0xFF, 0xFF, 0xFF)
};

[.]

for (int t = 0; t < 10000; t++) {
    for (int col = 0; col < MAXCOL; col++) {
        for (int row = 0; row < MAXROW - 1; row++) {
            firebuffer[row][col] = firebuffer[row+1][col] - rand() % 2;
            if (firebuffer[row][col] < 0) { firebuffer[row][col] = 0; }
        }
    }
}
```



0	0	0
n-2	n-1	n-1
n-1	n-1	n
n	n	n

und das für jeden Zeitschritt...  Animation

Pixelexfekte



```
#include <iostream>
#include <windows.h>

COLORREF COLOR_Arr[] = {
    RGB(0x0, 0x0, 0x0),
    //RGB(0x07, 0x07, 0x07),
    RGB(0x1F, 0x07, 0x07),
    RGB(0x2F, 0x0F, 0x07),
    RGB(0x47, 0x0F, 0x07),
    RGB(0x57, 0x17, 0x07),
    RGB(0x67, 0x1F, 0x07),
    RGB(0x77, 0x1F, 0x07),

[.]

for (int t = 0; t < 10000; t++) {
    for (int col = 0; col < MAXCOL; col++) {
        for (int row = 0; row < MAXROW - 1; row++) {
            firebuffer[row][col] = firebuffer[row+1][col] - rand() % 2;
            if (firebuffer[row][col] < 0) { firebuffer[row][col] = 0; }
        }
    }
}
```



n-3	n-1	n-2
n-2	n-1	n-1
n-1	n	n-1
n	n	n



1

Animation



- Menschliche Auge kann 12 Bilder/Sekunde (fps) unterscheiden
- Kino und TV: 24fps
- Computerspiele: meist 60fps

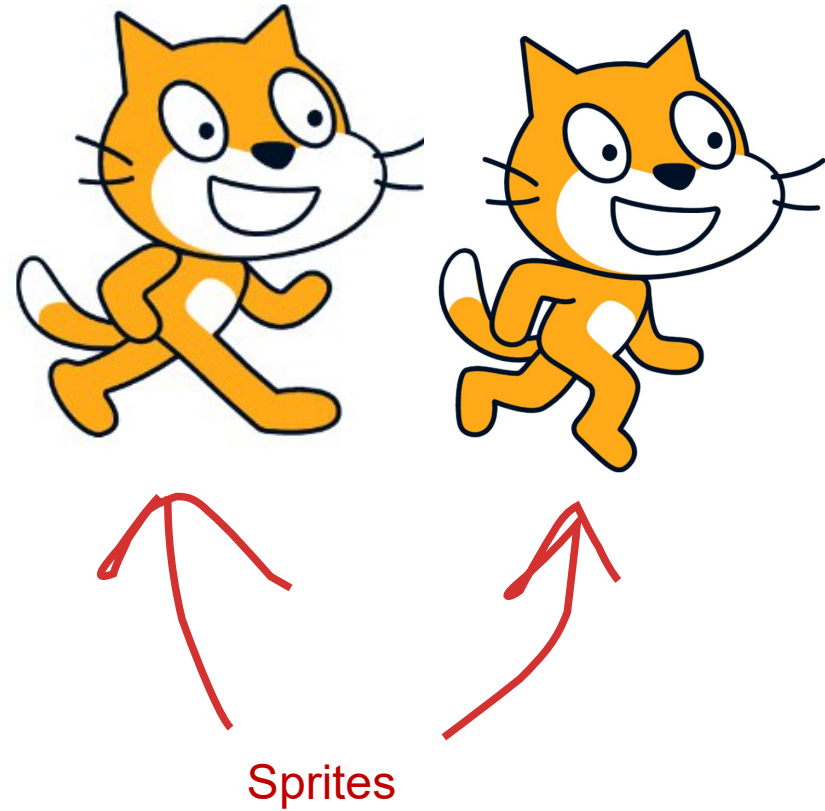


<https://www.wdrmaus.de/elefantenseite/elternseiten/selbermachen/anleitung-daumenkino.php>
<https://youtu.be/yFLa3ln16w0>

Vom Pixel zum Sprite



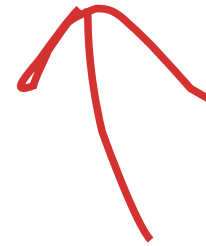
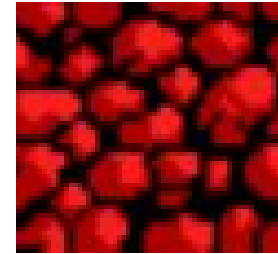
- Sprite ist ein Begriff der Computergrafik
- bezeichnet eine zweidimensionale Bitmap
- kann manipuliert und bewegt werden
- meist in eine größere Szene integriert
- -> 2D-Computerspiel



Vom Pixel zum Tile



- Tile ist ein Begriff der Computergrafik
- bezeichnet eine zweidimensionale Bitmap
- einfaches, wiederkehrendes Bild oder Muster
- für den Hintergrund oder Levelgestaltung
- -> 2D-Computerspiel



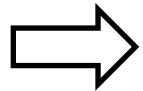
Tile



Vom Pixel zum Sprite & Tile



- Sprite ist ein Begriff der Computergrafik
- bezeichnet eine zweidimensionale Bitmap
- kann manipuliert und bewegt werden
- meist in eine größere Szene integriert
- Tile ist ein Begriff der Computergrafik
- bezeichnet eine zweidimensionale Bitmap
- einfaches, wiederkehrendes Bild oder Muster
- für den Hintergrund oder Levelgestaltung



2D-Computerspiel

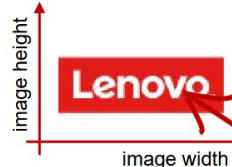


Commander Keen 1 (1990)
Erstes Side-Scrolling Spiel auf PC (Adaptive-Tile-Refresh-Technik)
https://www.retrogames.cz/play_471-DOS.php

[https://de.wikipedia.org/wiki/Sprite_\(Computergrafik\)](https://de.wikipedia.org/wiki/Sprite_(Computergrafik))
<https://github.com/freeciv/freeciv/tree/main/data/amplio>
<https://www.freeciv.org/>
<https://www.commander-keen.com/marooned-on-mars.php>







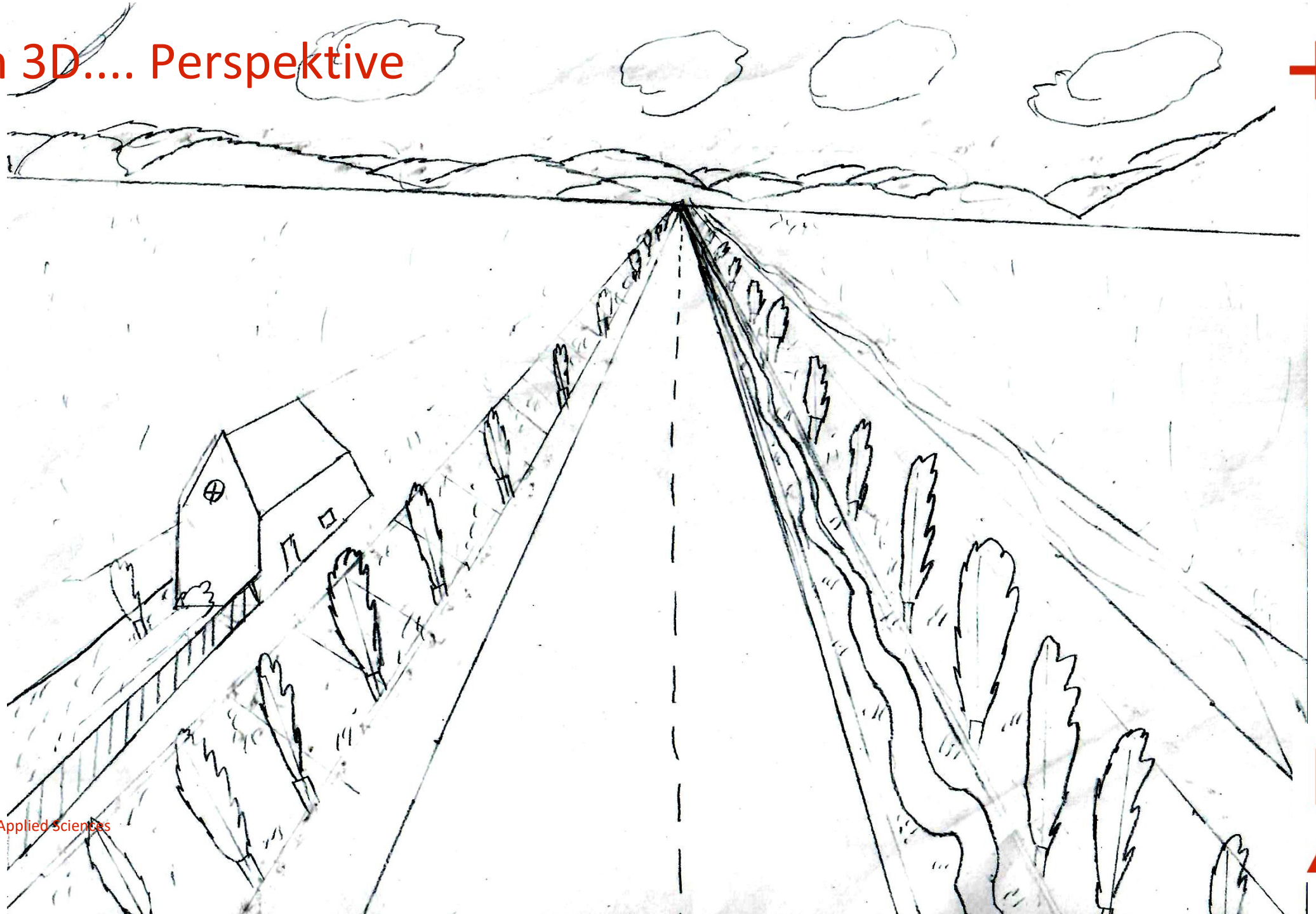
```
Datei ist BMP Datei
Datei ist BMP Datei
BMP ohne Kompression
BMP Width = 194
BMP Hight = 146
bits/pxl = 8
offset =
c. planar = 0
c. in rgb order = 0
Read 194 bytes... 256 colors...
```

```
C:\EigeneLokaleDaten\CPP\FileIO\x64\Debug\FileIO.exe (Prozess "7868") wurde mit Code
Drücken Sie eine beliebige Taste, um dieses Fenster zu schließen.
```


2D nach 3D?



2D nach 3D.... Perspektive



2.5D* → Early Racing Games



* „In the past, the term "2.5D" was more commonly used to refer to a variety of pseudo-3D graphical techniques that attempted to simulate three-dimensional graphics without the use of 3D polygons“

<https://www.giantbomb.com/25d/3015-660/>

<https://youtu.be/Bbb4w44bxqM>



2.5D → Early Racing Games



Karlsruhe University of Applied Sciences

<https://youtu.be/Bbb4w44bxqM>

<https://www.giantbomb.com/sprite-scaling/3015-7122/>

<https://www.giantbomb.com/super-scaler/3015-8389/>

Tausende von skalierten Sprites pro Sekunde!
(„Sega Super Scaler“)

2.5D → Early Racing Games

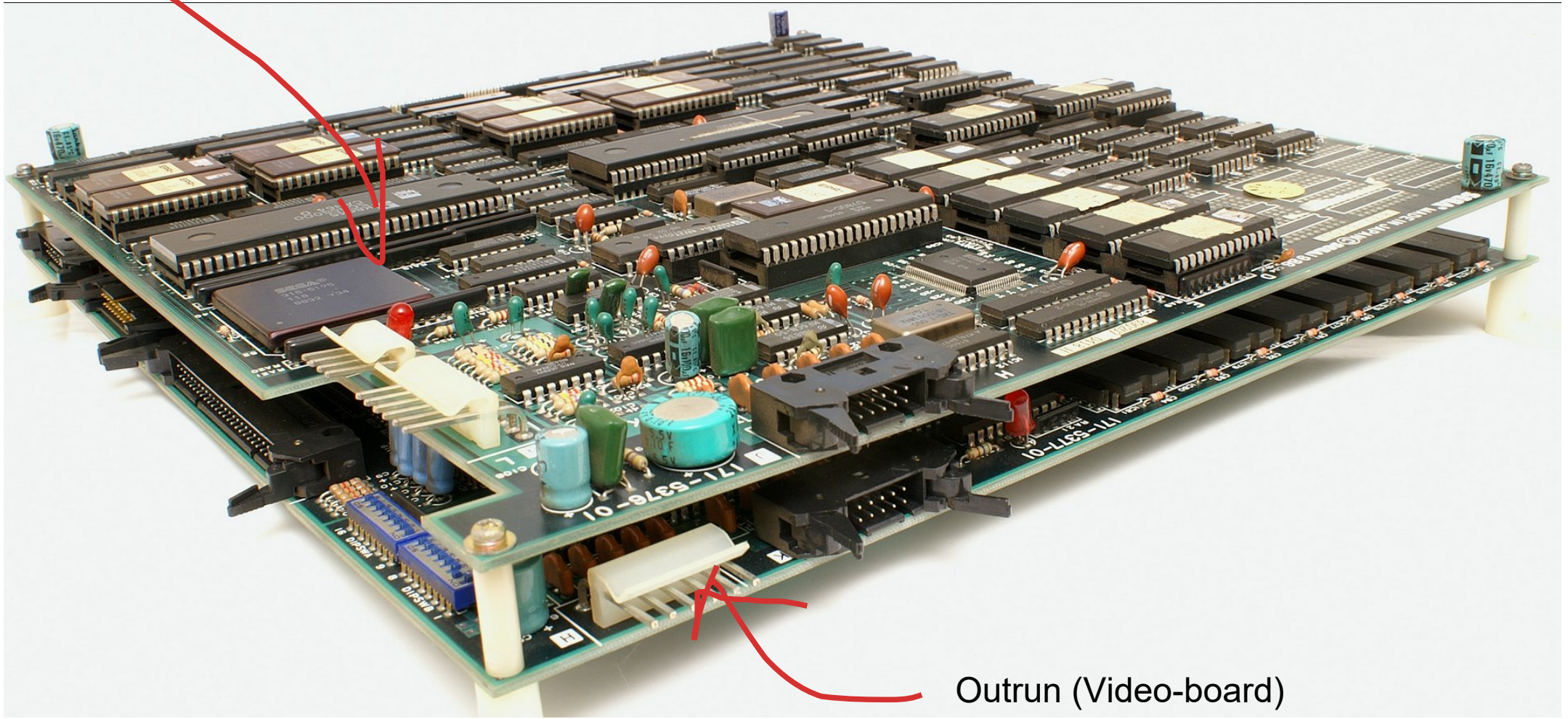


Out Run



Outrun – Deep Dive

Outrun (CPU-board)



Outrun (Video-board)

Outrun – Deep Dive

Road CPU
Motorola 68000
10MHz

Road

- generating
- manipulating

EPROMs: graphic for
each road

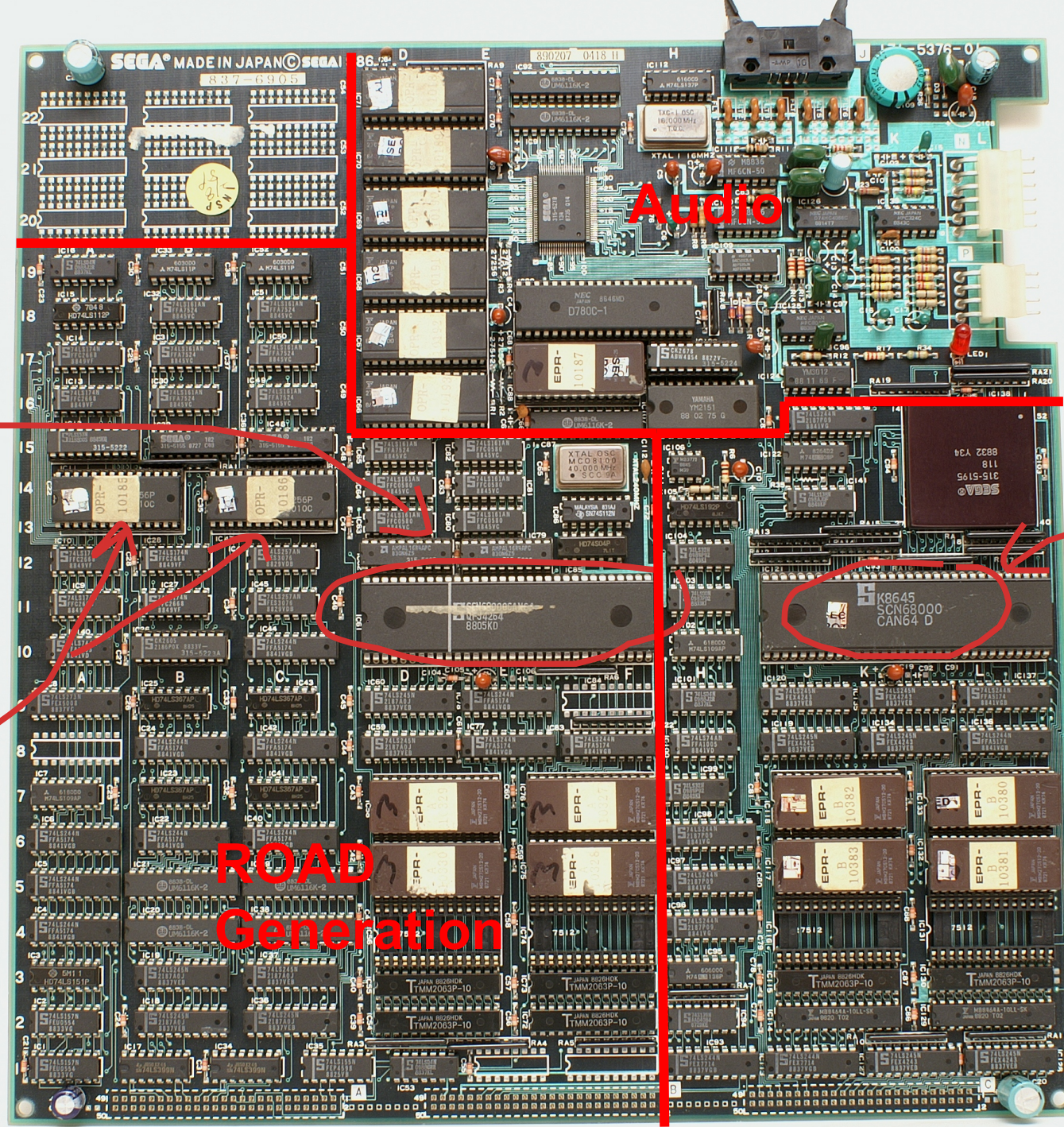
**ROAD
Generation**

Audio

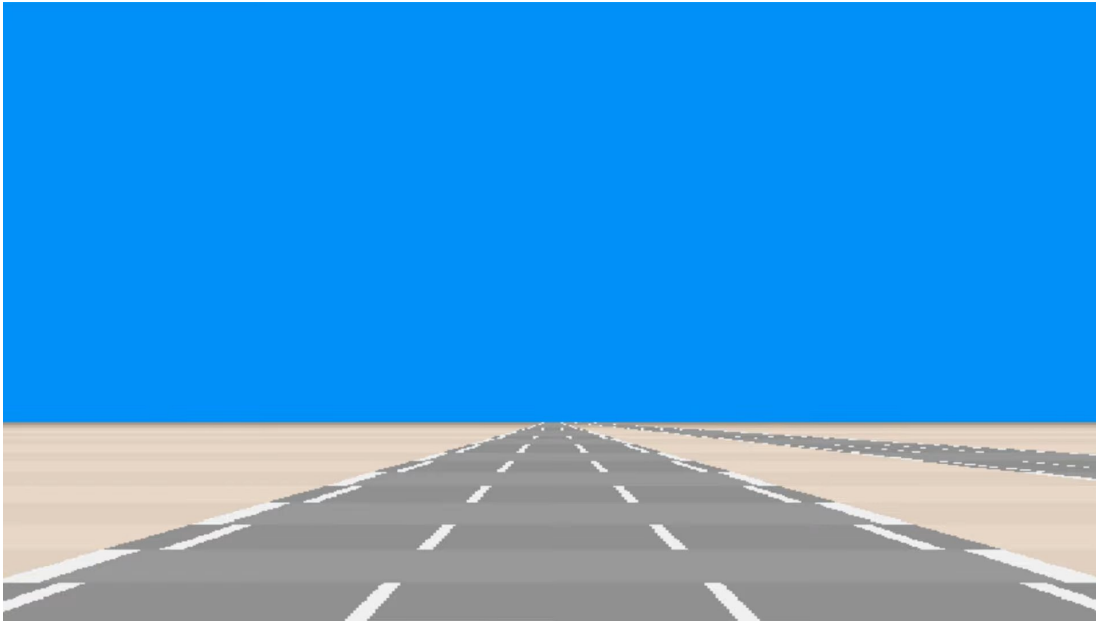
Master CPU
Motorola 68000
10MHz

Game

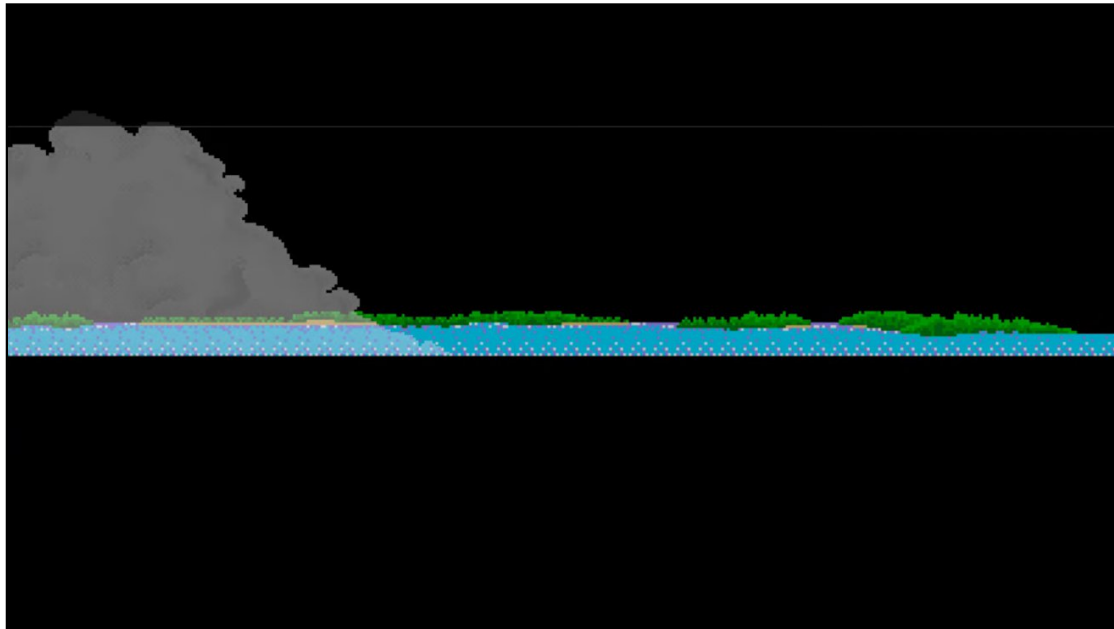
- logic
- environment



Road Rendering



2 independent scrolling tile map layers



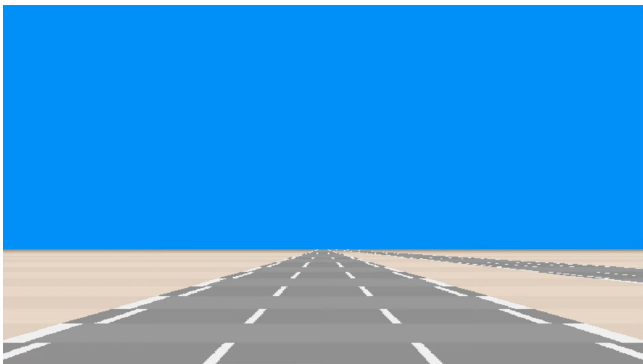
Sprite Rendering and Scaling



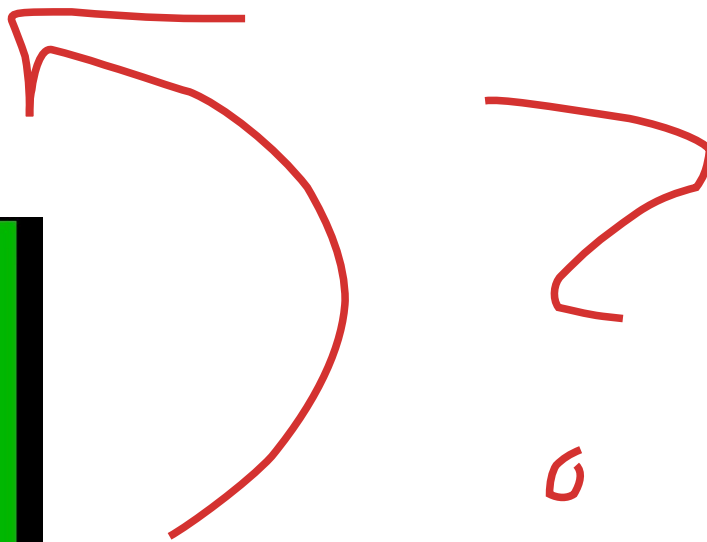
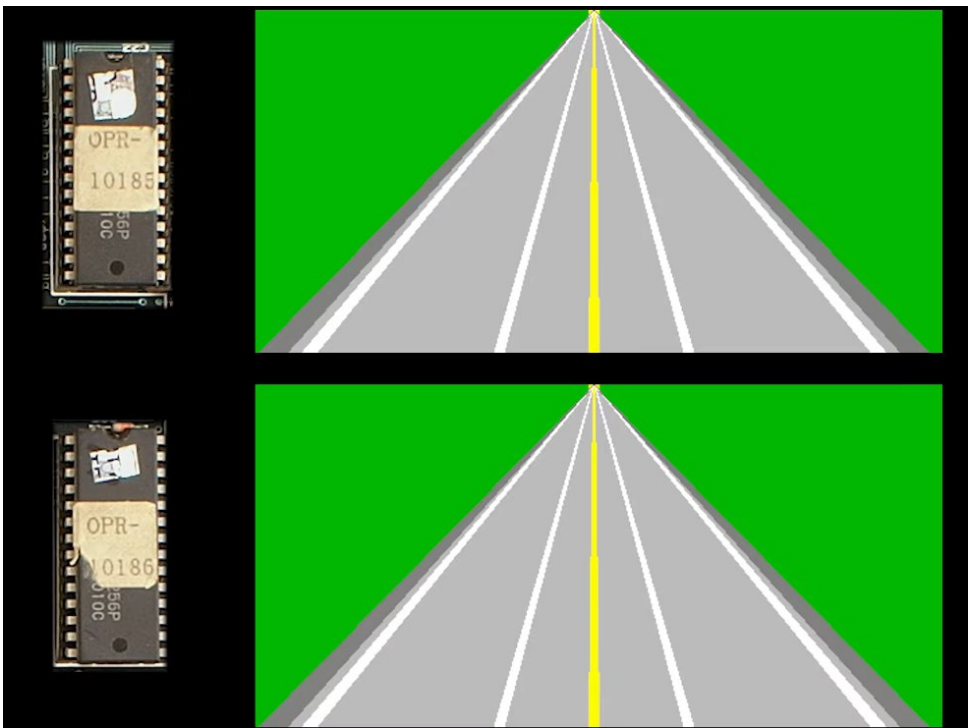
text overlay layer



Road Rendering



Outrun – Deep Dive



Road Rendering

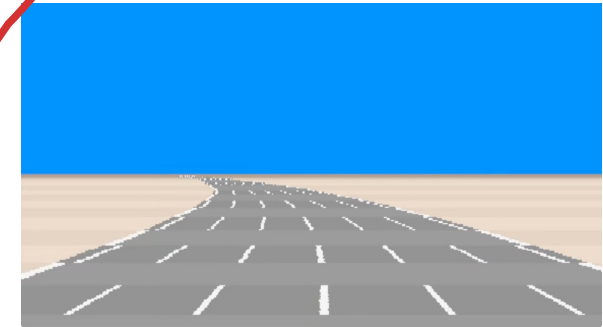


Outrun – Deep Dive

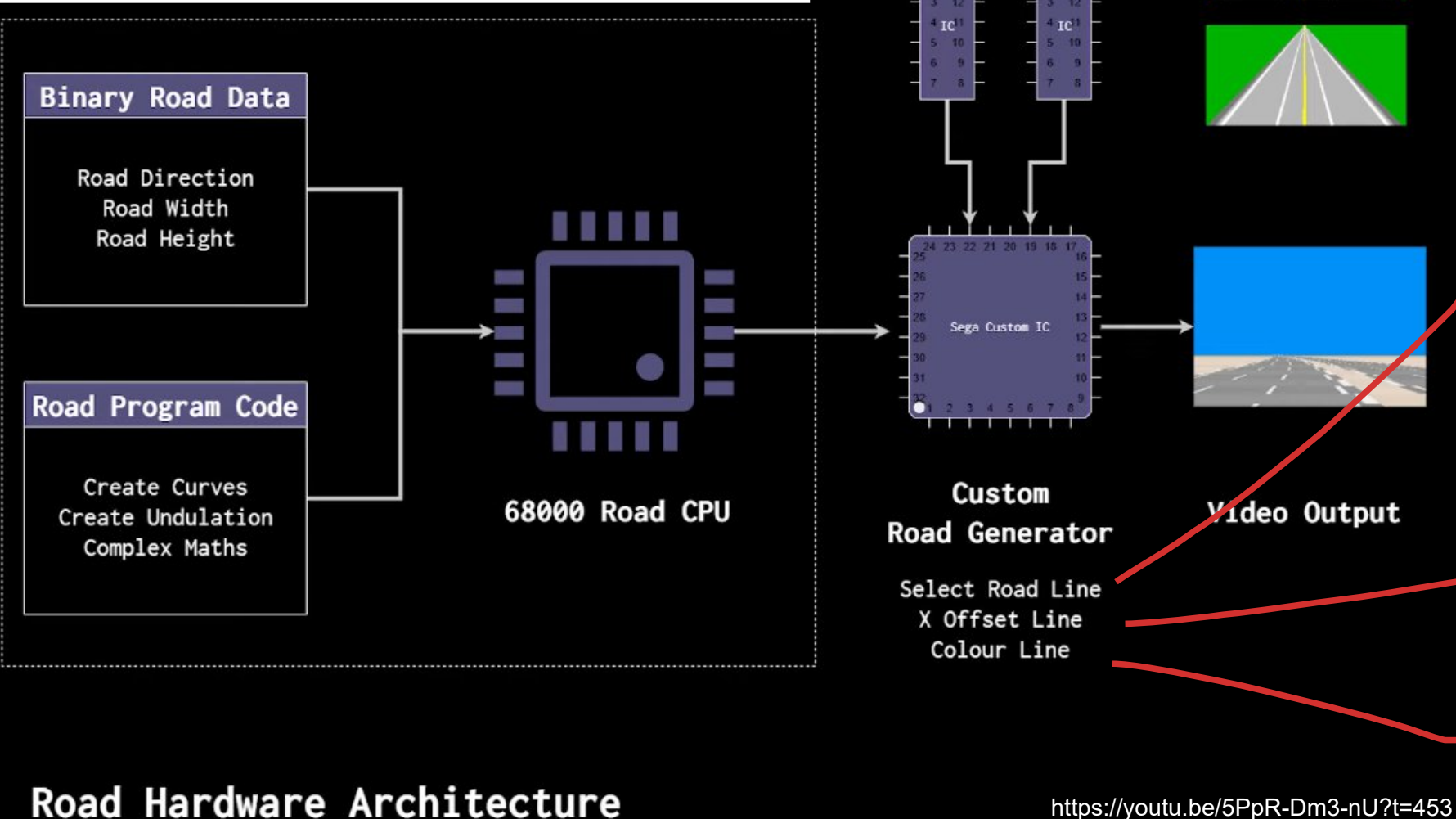
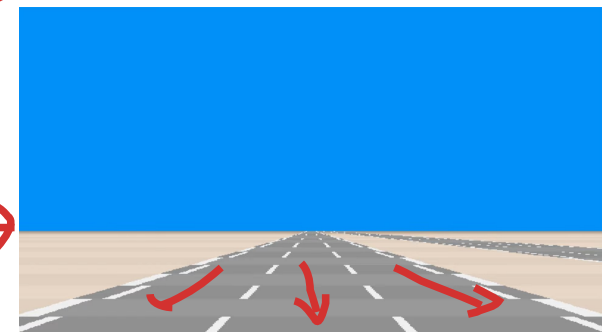
Undulation



Curvature



Movement



2.5D → MODE 7

+

- Funktion zur Grafikdarstellung (Nintendo)
- SNES und Game Boy Advance (~1991-2000)
- Texture-Mapping-Modus:
2D Grafik -> dynamische 3D Ebene

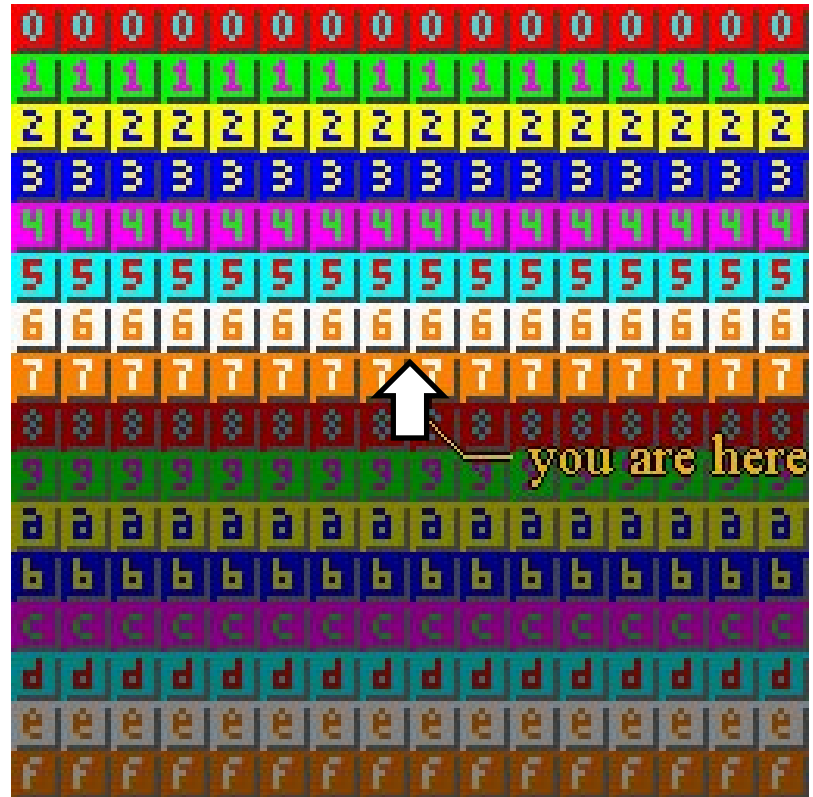


K
A

2.5D → MODE 7



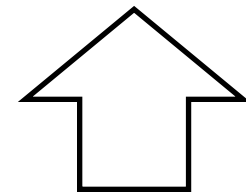
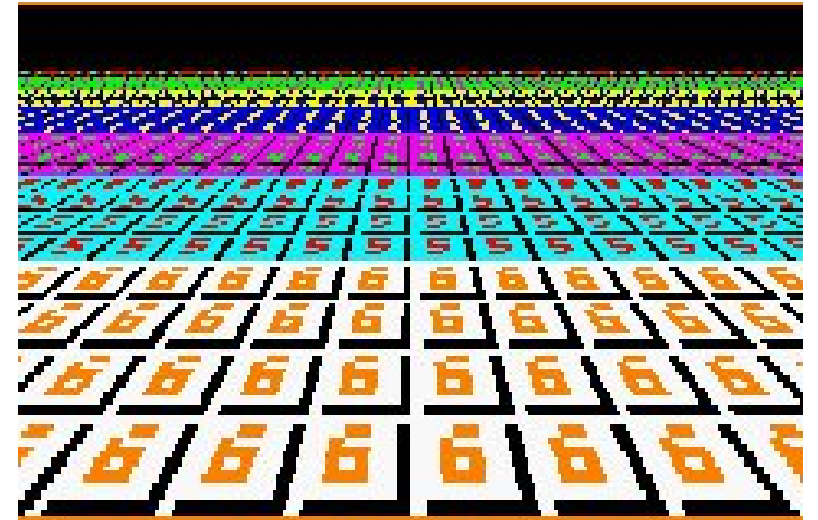
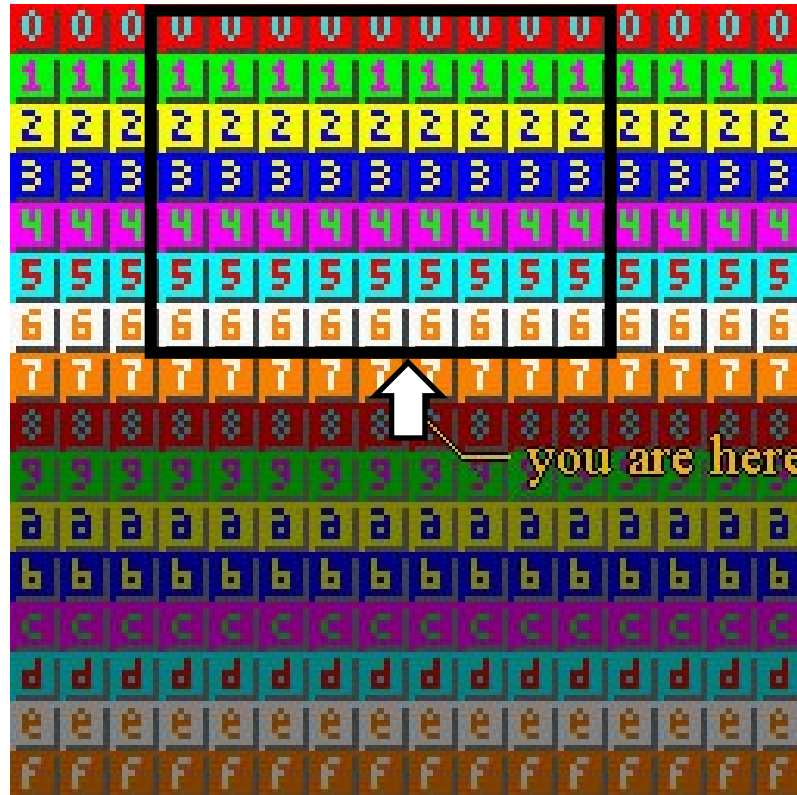
- Funktion zur Grafikdarstellung (Nintendo)
- SNES und Game Boy Advance (~1991-2000)
- Texture-Mapping-Modus:
2D Grafik -> dynamische 3D Ebene



2.5D → MODE 7

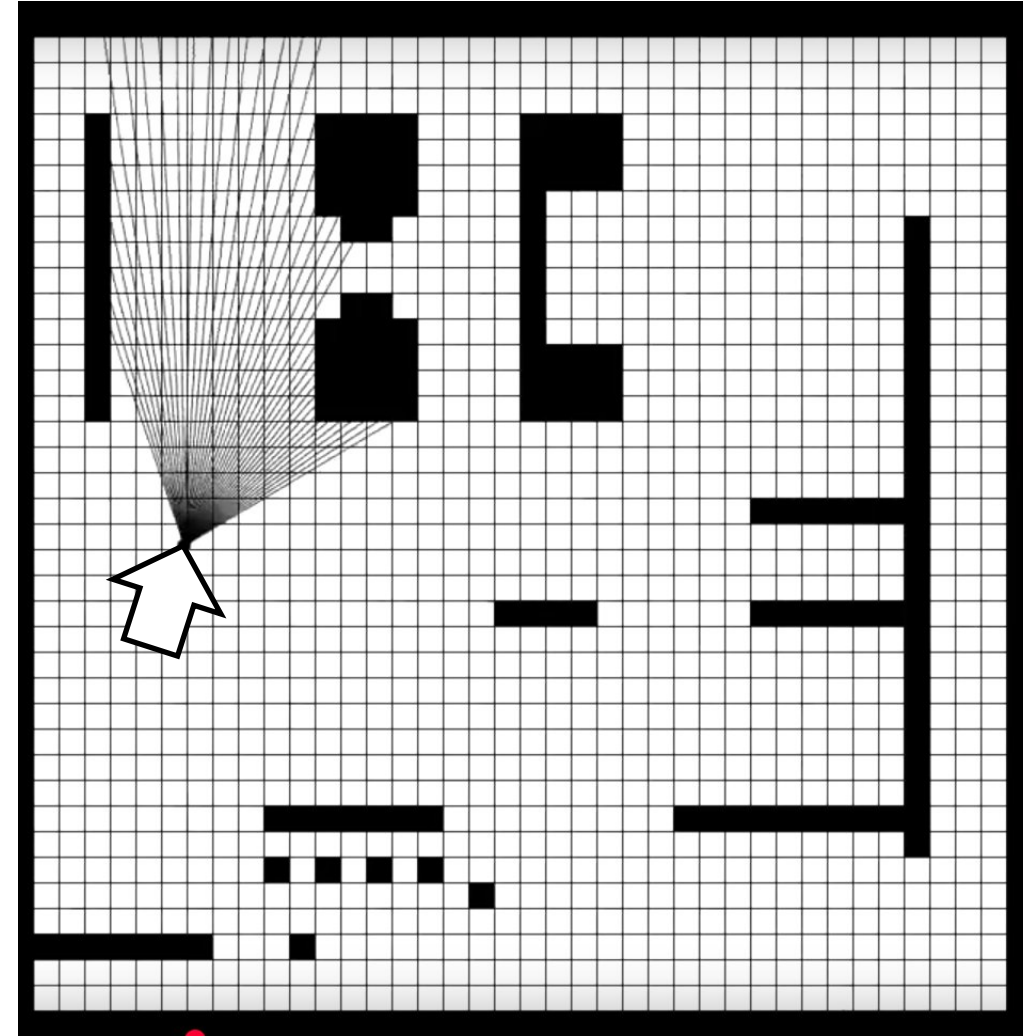


- Funktion zur Grafikdarstellung (Nintendo)
- SNES und Game Boy Advance (~1991-2000)
- Texture-Mapping-Modus:
2D Grafik -> dynamische 3D Ebene



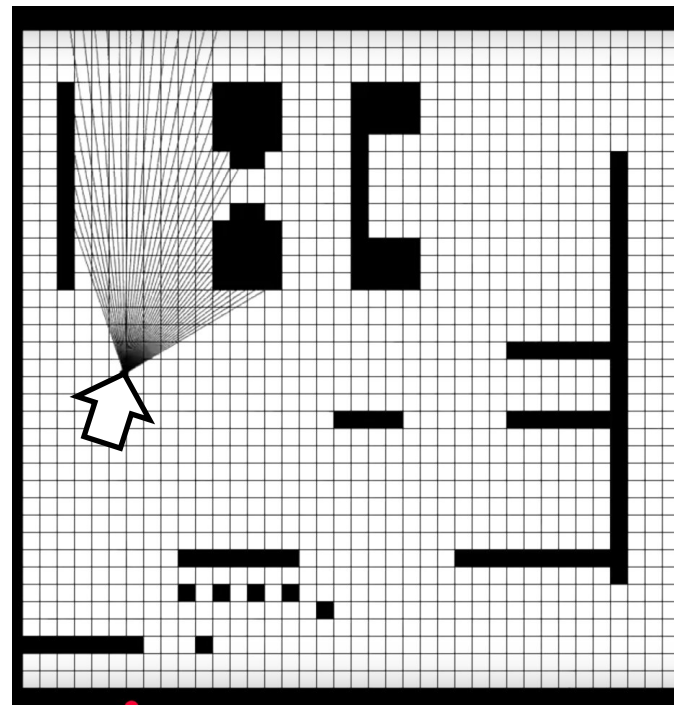
2.5D → Raycasting

- Texture-Mapping-Modus:
2D Grafik -> dynamische 3D Ebene
- Ausgehend von einer Position & Orientierung in einer 2D Karte
 - Aussenden von Strahlen
 - Messen der Distanz d zum nächsten Objekt



2.5D → Raycasting

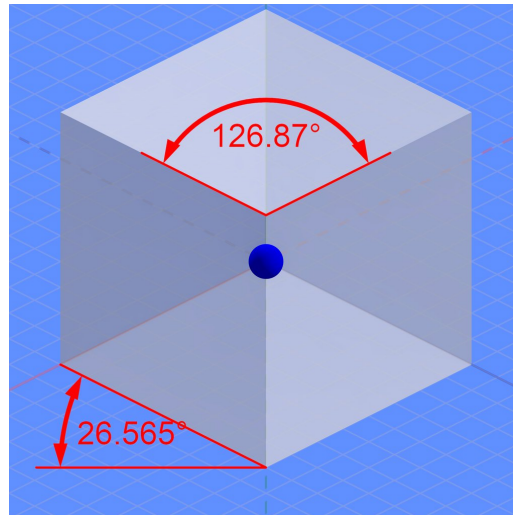
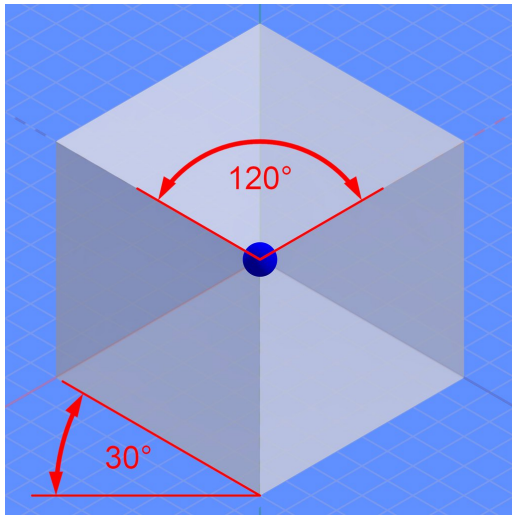
- Texture-Mapping-Modus:
2D Grafik -> dynamische 3D Ebene
- Ausgehend von einer Position & Orientierung in einer 2D Karte
 - Aussenden von Strahlen
 - Messen der Distanz d zum nächsten Objekt
 - Darstellen des Objekts als Funktion von d



2.5D → Isometrische Perspektive (Isometric View)



- Darstellungsform die 3D Eindruck erzeugen soll
- Isometrie mathematisch: alle Raumachsen 120° zueinander -> Verzerrung bzw. Längenverhältnisse über alle drei Achsen gleich
- In Computerspielen Perspektive nicht immer isometrisch



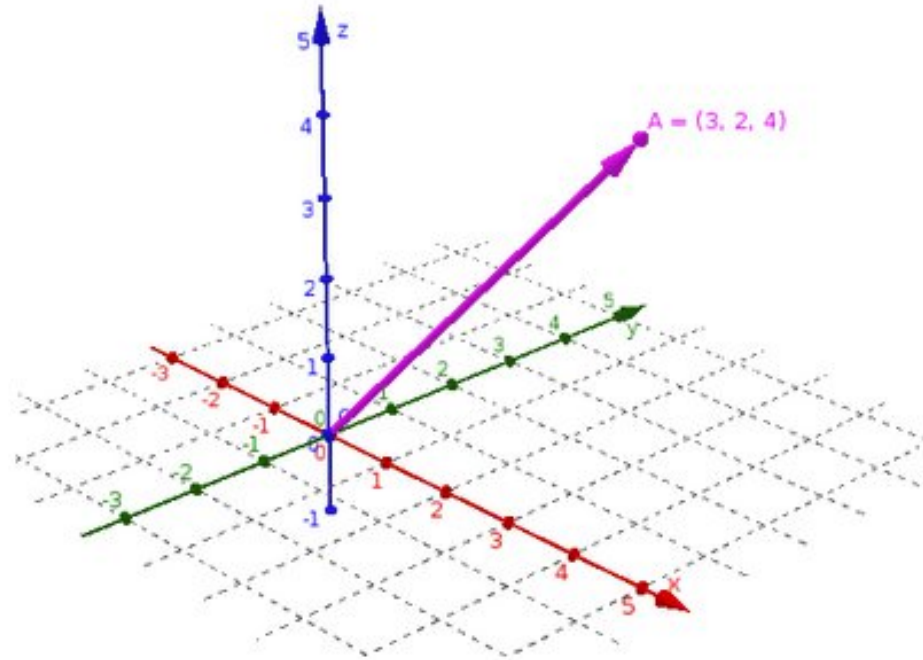
[https://de.wikipedia.org/wiki/Sprite_\(Computergrafik\)](https://de.wikipedia.org/wiki/Sprite_(Computergrafik))
<https://github.com/freeciv/freeciv/tree/main/data/amplio>
<https://www.freeciv.org/>



3D → Punkte

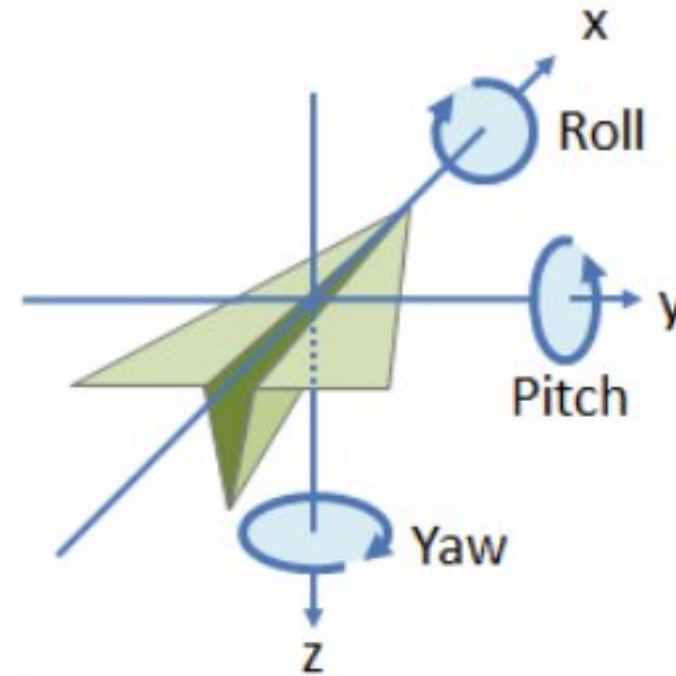


- Beschreiben von 3D Punkten im Raum mit 3D Vektoren



3D → Punkte

- Beschreiben von 3D Punkten im Raum mit 3D Vektoren
- Lineare Transformationen
 - Transformationsmatrix
 - Rotationsmatrix

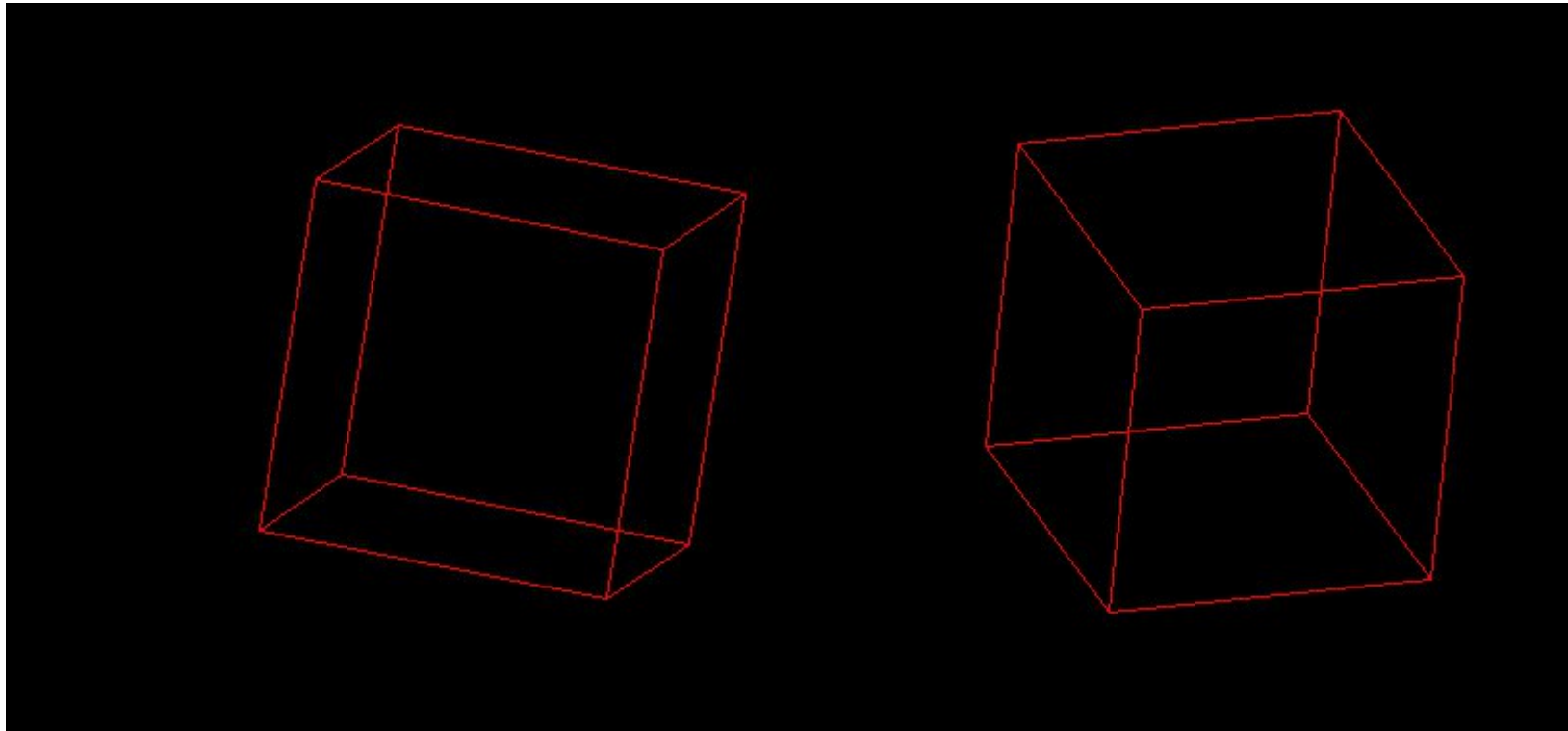


$$R = R_z(\alpha) R_y(\beta) R_x(\gamma) = \begin{bmatrix} \cos \alpha & \overset{\text{yaw}}{-\sin \alpha} & 0 \\ \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \beta & \overset{\text{pitch}}{0} & \sin \beta \\ 0 & 1 & 0 \\ -\sin \beta & 0 & \cos \beta \end{bmatrix} \begin{bmatrix} 1 & \overset{\text{roll}}{0} & 0 \\ 0 & \cos \gamma & -\sin \gamma \\ 0 & \sin \gamma & \cos \gamma \end{bmatrix}$$
$$= \begin{bmatrix} \cos \alpha \cos \beta & \cos \alpha \sin \beta \sin \gamma - \sin \alpha \cos \gamma & \cos \alpha \sin \beta \cos \gamma + \sin \alpha \sin \gamma \\ \sin \alpha \cos \beta & \sin \alpha \sin \beta \sin \gamma + \cos \alpha \cos \gamma & \sin \alpha \sin \beta \cos \gamma - \cos \alpha \sin \gamma \\ -\sin \beta & \cos \beta \sin \gamma & \cos \beta \cos \gamma \end{bmatrix}$$

3D → 3D Projektion auf XY-Ebene

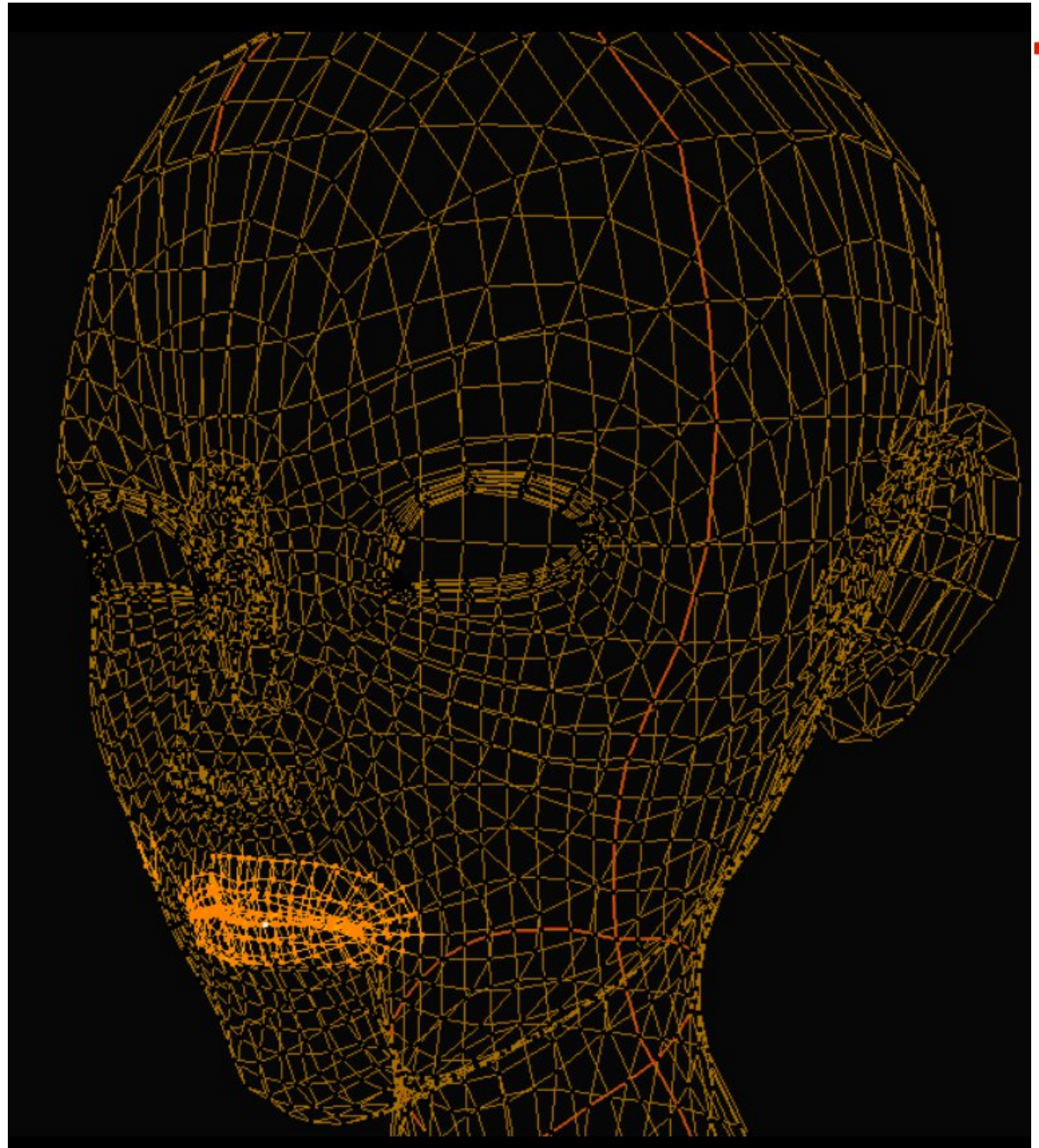
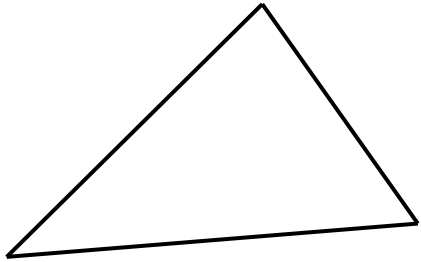


- Beschreiben von 3D Punkten im Raum mit 3D Vektoren
- Lineare Transformationen
 - Transformationsmatrix
 - Rotationsmatrix
- Projektion auf XY- Ebene



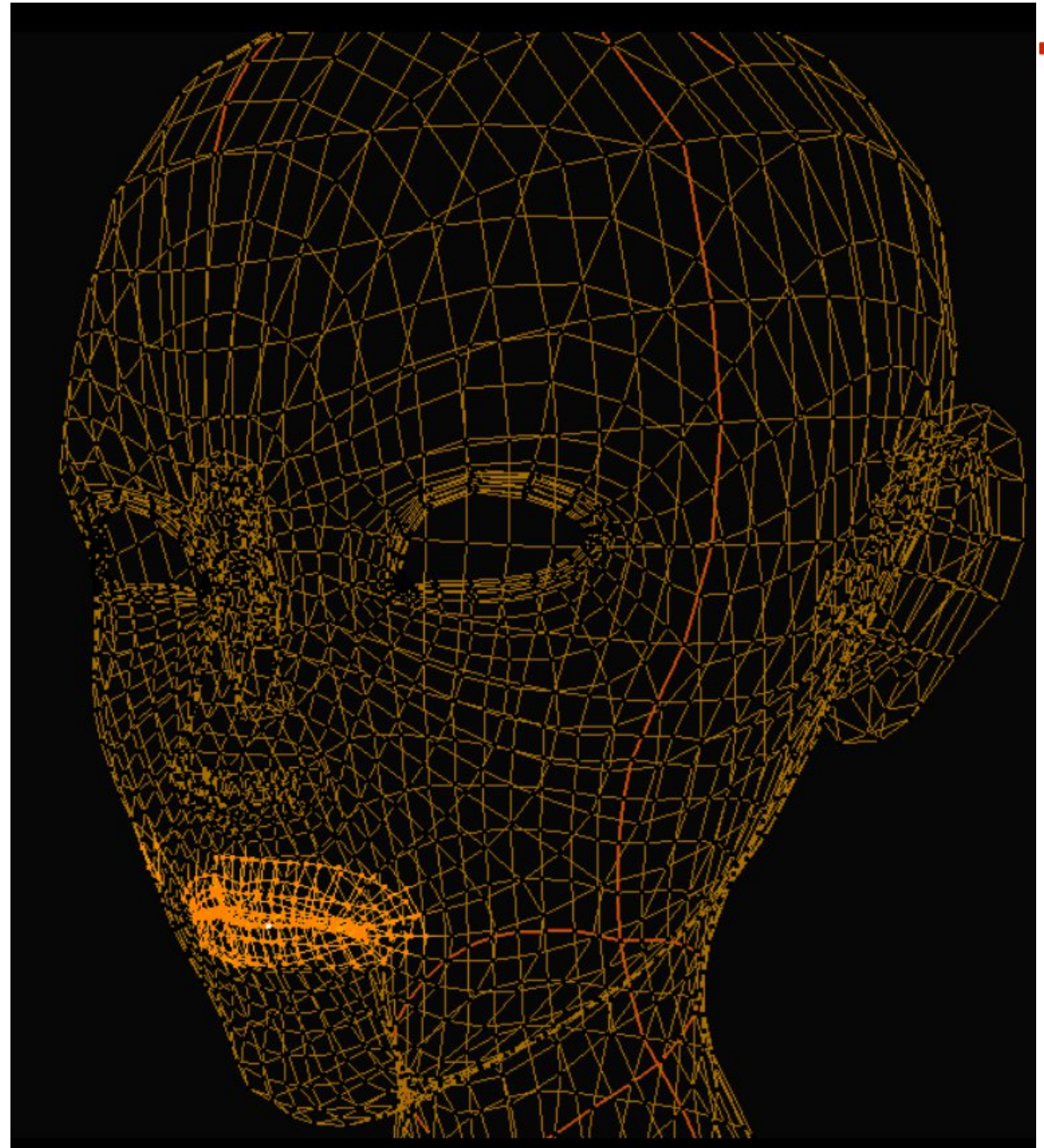
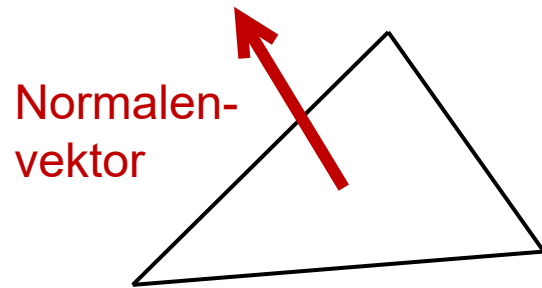
3D → Polygone

- 3D Modelle bestehen aus vielen Vielecken
- zumeist werden Dreiecke benutzt
- auch als Drahtgittermodell bezeichnet



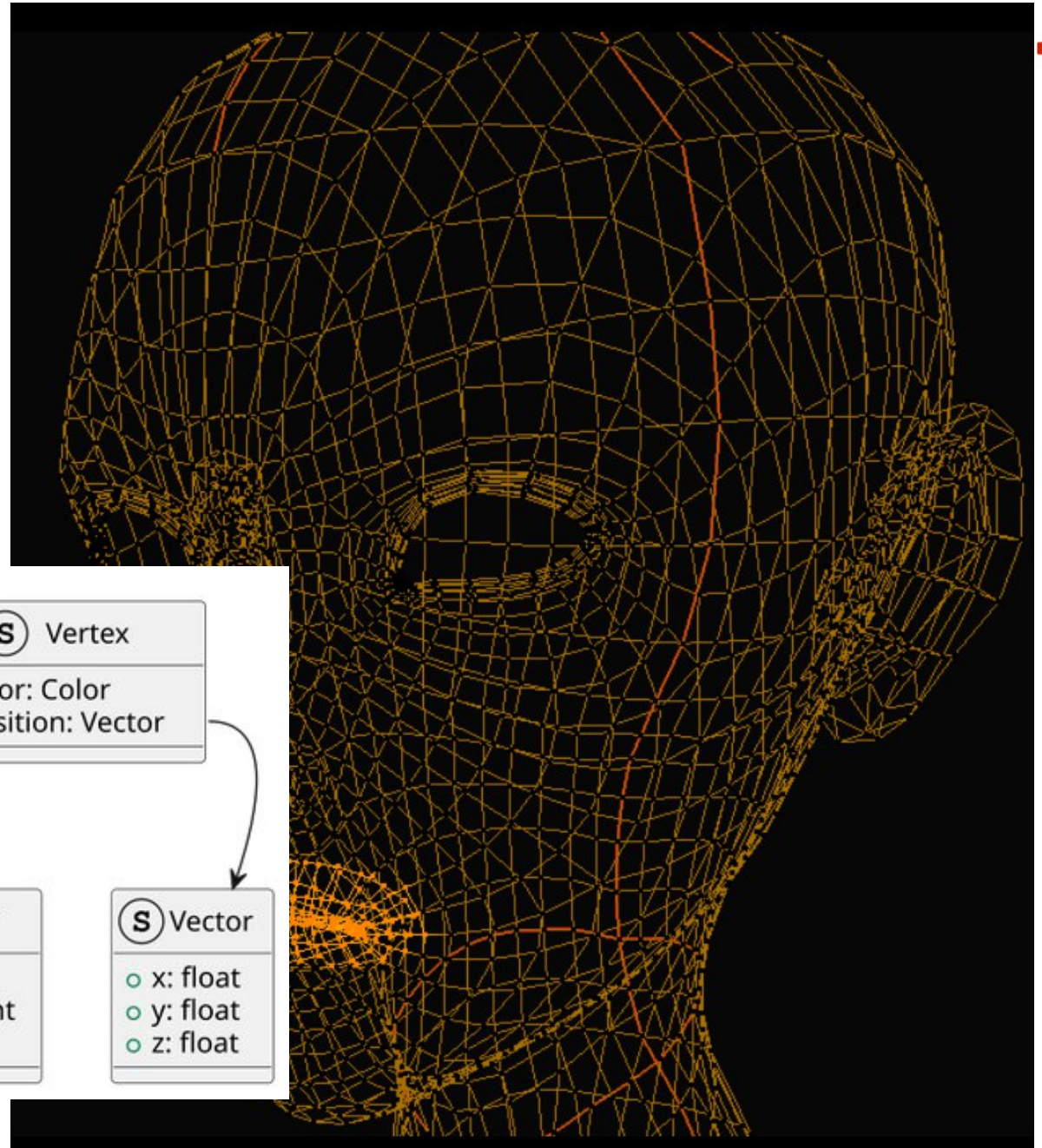
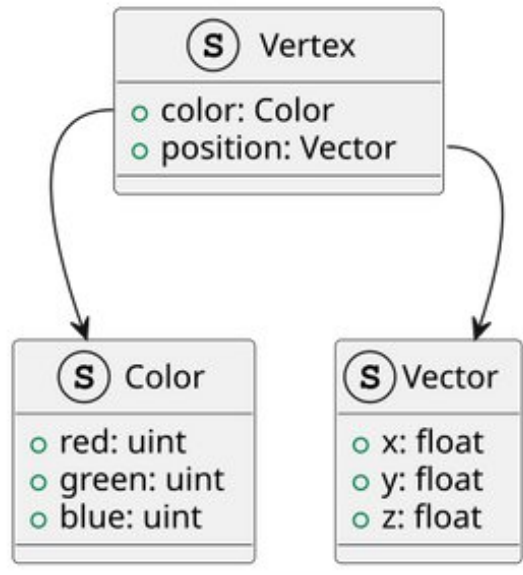
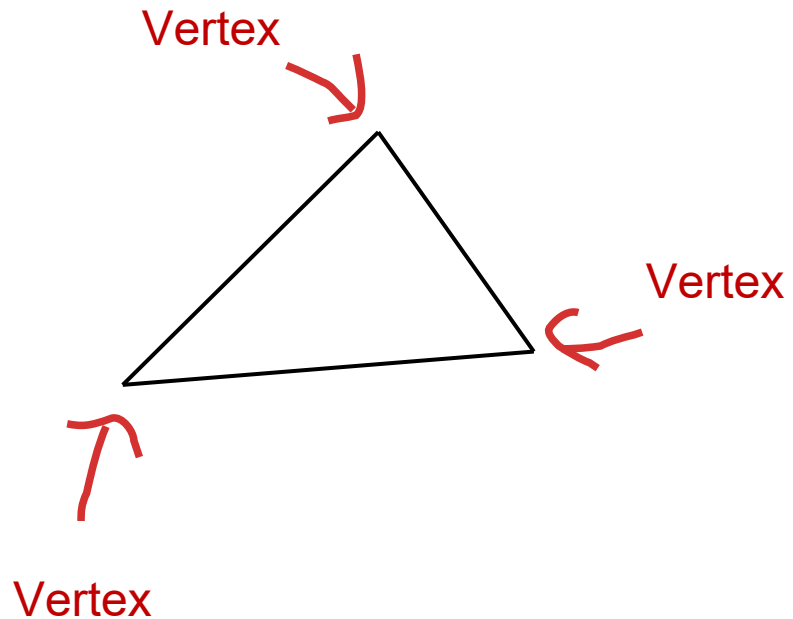
3D → Polygone

- 3D Modelle bestehen aus vielen Vielecken
- zumeist werden Dreiecke benutzt
- auch als Drahtgittermodell bezeichnet



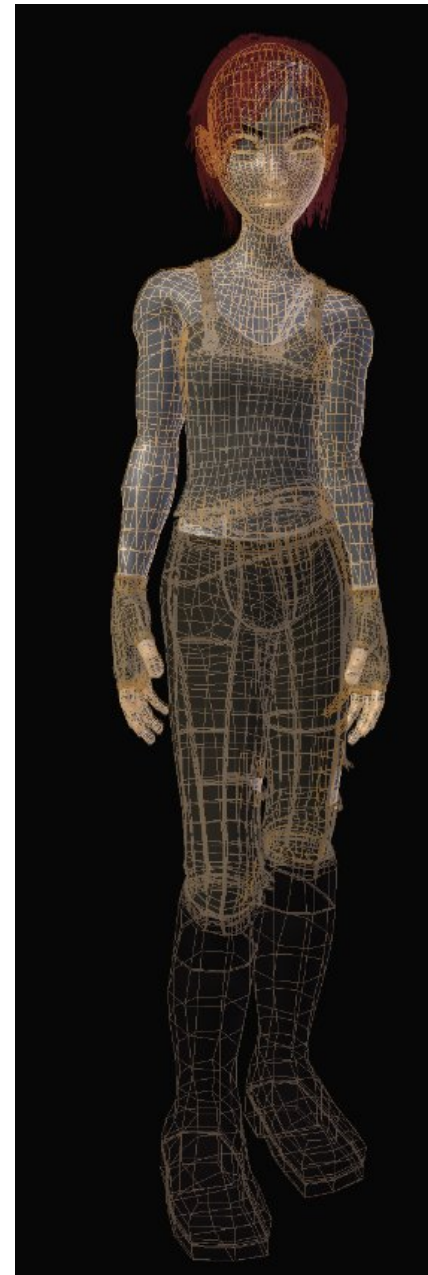
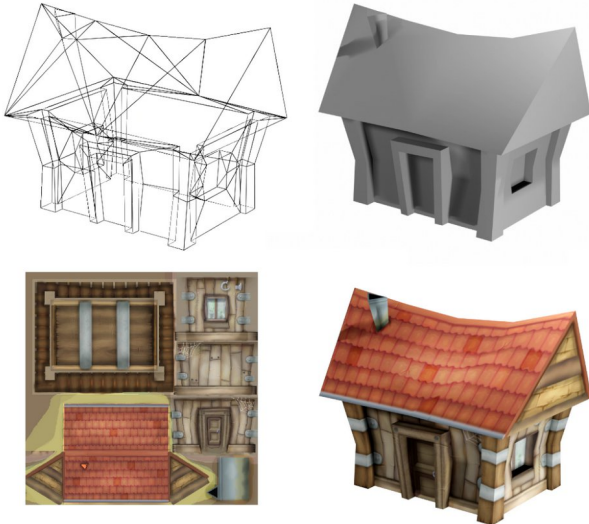
3D → Polygone

- 3D Modelle bestehen aus vielen Vielecken
- zumeist werden Dreiecke benutzt
- auch als Drahtgittermodell bezeichnet



3D → Polygone

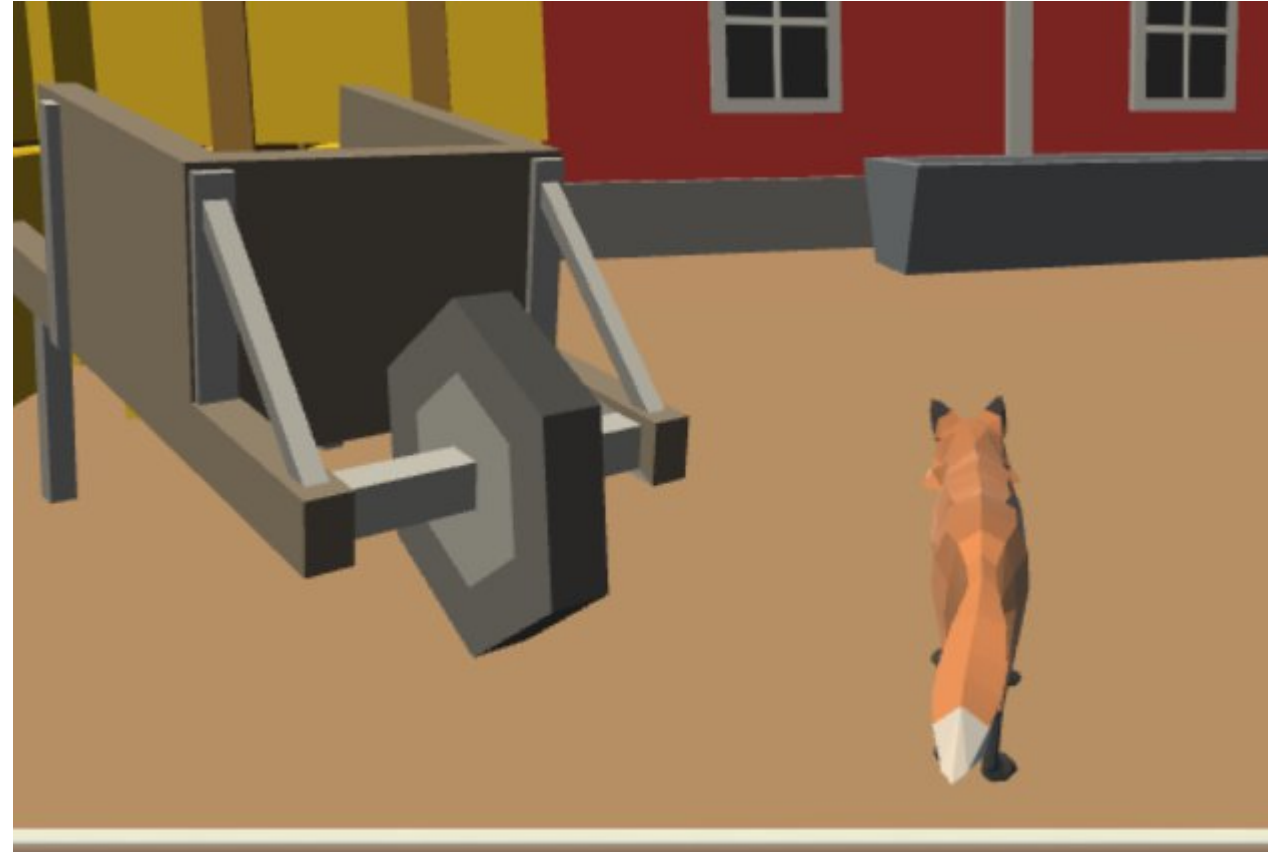
- 3D Modelle bestehen aus vielen Vielecken
- zumeist werden Dreiecke benutzt
- auch als Drahtgittermodell bezeichnet
- anschließend werden Texturen auf das 3D-Modell gemapped
- Hierzu wird jedem Punkt des Gitters eine Texturkoordinate (UV-Koordinaten) zugeordnet.



3D → Polygone



- Beispiel rudimentärer Polygonengrafik:
- z.B. Rad mit sehr wenigen Polygonen



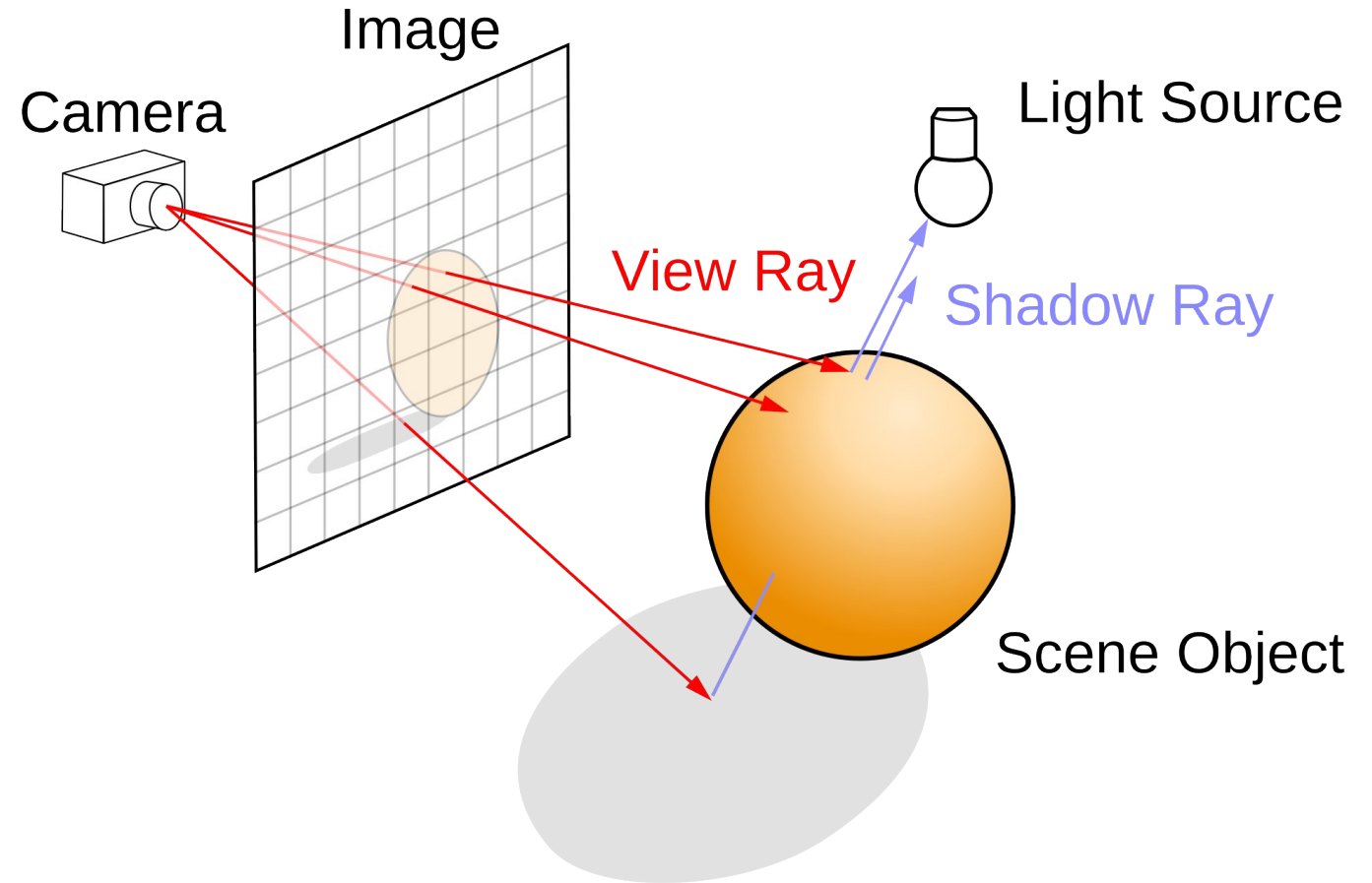
<https://poki.com/de/g/fox-simulator-3d>



3D → Raytracing



- Raytracing-Algorithmus ermöglicht Darstellung einer 3D-Szene.
- Der Weg von Auge durch die Szene zur der/den Lichtquellen wird simuliert.



3D → Raytracing



- Raytracing-Algorithmus ermöglicht Darstellung einer 3D-Szene.
- Der Weg von Auge durch die Szene zur der/den Lichtquellen wird simuliert.
- Zusätzlich der Weg von Lichtstrahlen durch die Szene zum Auge (Path Tracing)



<https://de.wikipedia.org/wiki/Raytracing>
[https://en.wikipedia.org/wiki/Ray_tracing_\(graphics\)](https://en.wikipedia.org/wiki/Ray_tracing_(graphics))

Raytracing-Verfahren	Schatten	Lichtreflexion / -brechung / -streuung	Beleuchtung	Beispielbild
Nur Verdeckungsrechnung	Keine	Keine	Keine	
Simulation von Schatten (mit nur einem Schattenstrahl)	Nur harte Schatten	Keine	Nur direkte Beleuchtung	
Rekursives Raytracing	Nur harte Schatten	Nur bei spiegelnden / brechenden Oberflächen	Nur direkte Beleuchtung oder Spiegelung / Brechung	
Diffuses Raytracing	Vollständig	Nur bei spiegelnden / brechenden Oberflächen	Nur direkte Beleuchtung oder Spiegelung / Brechung	
Path Tracing (und nachfolgende Verfahren)	Vollständig	Vollständig (auch diffus)	Vollständig (Globale Beleuchtung)	





C++

Simple DirectMedia Layer (SDL)

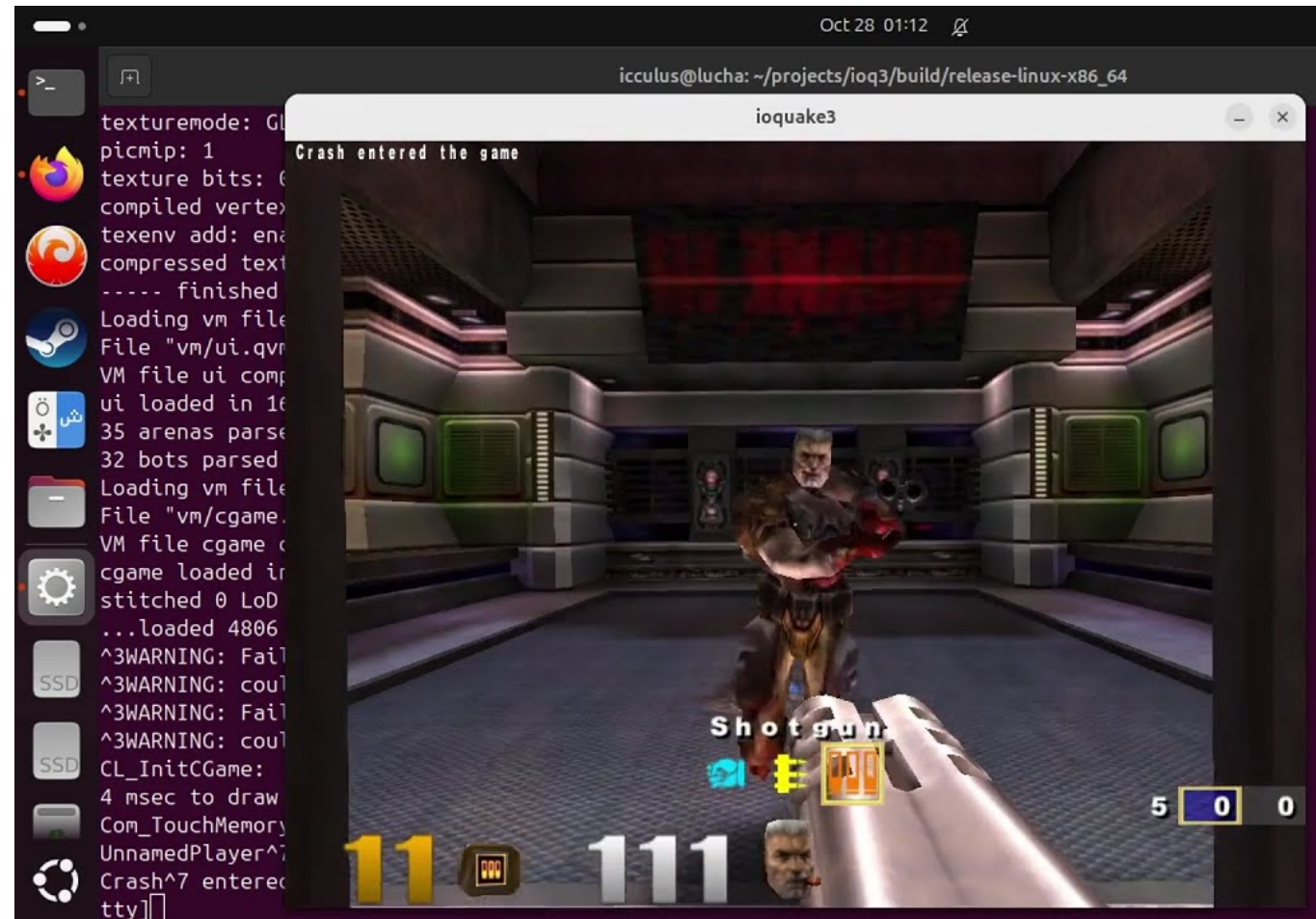


- current version 3.0 (also called SDL3)
- cross-platform development library
- low level access to
 - graphics hardware (via OpenGL/Direct3D/Metal/Vulkan)
 - audio
 - keyboard
 - mouse
 - joystick

<https://wiki.libsdl.org/SDL3/FrontPage>

<https://youtu.be/i3yVqWYFbCE> (IOQuake3 SDL2 -> SDL3)

<https://github.com/ioquake/ioq3>



Simple DirectMedia Layer (SDL)

- current version 3.0 (also called SDL3)
- cross-platform development library
- low level access to
 - graphics hardware (via OpenGL/Direct3D/Metal/Vulkan)
 - audio
 - keyboard
 - mouse
 - joystick
- supports
 - Windows
 - macOS
 - Linux
 - iOS, and Android
 - several other platforms
- written in C and works
 - natively with C++
 - bindings for C#, Python, Rust (and others)

<https://wiki.libsdl.org/SDL3/FrontPage>

<https://youtu.be/i3yVqWYFbCE> (IOquake3 SDL2 -> SDL3)

<https://github.com/ioquake/ioq3>

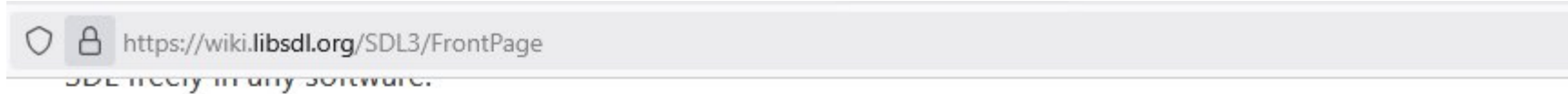


<https://youtu.be/XfZ6WrV5Z7Y?t=524>

<https://fr.techtribune.net/tech-today/de-quel-type-de-pc-avez-vous-besoin/215755/>

<https://commons.wikimedia.org/>

K
A



This is the SDL wiki. SDL's main website is [libsdl.org](https://www.libsdl.org/).

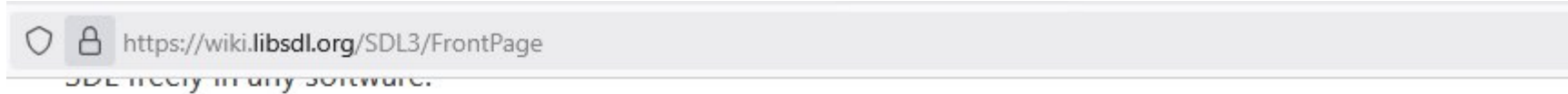
Getting and installing SDL

- [Source Code](#): How to download the source code to SDL.
- [Installation](#): How to install SDL on your platform of choice and link your program against it.

API Documentation

- [API by category](#)
- [API quick reference](#)
- [Complete API index](#)

<https://wiki.libsdl.org/SDL3/FrontPage>



This is the SDL wiki. SDL's main website is [libsdl.org](https://www.libsdl.org/).

Getting and installing SDL

- [Source Code](#): How to download the source code to SDL.
- [Installation](#): How to install SDL on your platform of choice and link your program against it.

API Documentation

- [API by category](#)
- [API quick reference](#)
- [Complete API index](#)



<https://wiki.libsdl.org/SDL3/FrontPage>

SDL Informationen

https://wiki.libsdl.org/SDL3/FrontPage

This is the SDL wiki. SDL's main website is [libsdl.org](https://www.libsdl.org/).

Getting and installing SDL

- [Source Code](#): How to download the source code to SDL.
- [Installation](#): How to install SDL on your platform of choice and link your program against it.

API Documentation

- [API by category](#) 
- [API quick reference](#)
- [Complete API index](#)

<https://wiki.libsdl.org/SDL3/FrontPage>

SDL 3.0 API by Category

Basics

View information and functions related to... View the header	
Application entry points	SDL_main.h
Initialization and Shutdown	SDL_init.h
Configuration Variables	SDL_hints.h
Object Properties	SDL_properties.h
Error Handling	SDL_error.h
Log Handling	SDL_log.h
Assertions	SDL_assert.h
Querying SDL Version	SDL_version.h

Video

View information and functions related to... View the header	
Display and Window Management	SDL_video.h
2D Accelerated Rendering	SDL_render.h
Pixel Formats and Conversion Routines	SDL_pixels.h
Blend modes	SDL_blendmode.h
Rectangle Functions	SDL_rect.h

Pixel mit SDL



```
#include <iostream>
#include <SDL3/SDL.h>
using namespace std;

int main()
{
    cout << "Just a point\n";

    // 1) Init
    if (SDL_Init(SDL_INIT_VIDEO)) {
        cout << "Window initialization is successfull.\n";
    }
    else {
        cout << SDL_GetError();
        SDL_Quit();
        return -1;
    }

    // 2) create window
    SDL_Window* window = nullptr;
    window = SDL_CreateWindow("SDL3 Start", 1280, 720,
        SDL_WINDOW_VULKAN | SDL_WINDOW_HIGH_PIXEL_DENSITY);
    if (window == nullptr) {
        cout << "ERROR creating window\n";
        SDL_Quit();
        return -1;
    }
    else cout << "Window created successfully.\n";
```

```
    // 3 Write to surface directly
    // You may not combine this with 3D or the rendering API on this window.
    SDL_Surface* surface = SDL_GetWindowSurface(window);
    SDL_ClearSurface(surface, 0, 0, 0, 0);
    if (SDL_WriteSurfacePixel(surface, 10, 10, 255, 0, 0, 1)) {
        cout << "Pixel written successfully\n";
    }
    else {
        cout << "ERROR: Pixel\n";
    }
    SDL_UpdateWindowSurface(window);

    SDL_Delay(5000); // 5 Sekunden warten

    // 4) Clean up & close
    SDL_DestroyWindow(window);
    SDL_Quit();
}
```

Pixel mit SDL



```
#include <iostream>
#include <SDL3/SDL.h>
using namespace std;
```

```
int main()
{
```

```
    cout << "Just a point\n";
```

```
    // 1) Init
```

```
    if (SDL_Init(SDL_INIT_VIDEO)) {
        cout << "Window initialization failed\n";
    }
```

```
    else {
        cout << SDL_GetError();
        SDL_Quit();
        return -1;
    }
```

```
    // 2) create window
```

```
    SDL_Window* window = SDL_CreateWindow(
        "Just a point",
        SDL_WINDOWPOS_CENTERED,
        SDL_WINDOWPOS_CENTERED,
        640, 480,
        SDL_WINDOW_VULKAN);
    if (window == nullptr) {
        cout << "ERROR creating window\n";
        SDL_Quit();
        return -1;
    }
```

```
    else cout << "Window created successfully\n";
```

```
    // 3 Write to surface directly
```

```
    // You may not combine this with 3D or the rendering API on this window.
```

```
    SDL_Surface* surface = SDL_GetWindowSurface(window);
```

```
    SDL_ClearSurface(surface, 0, 0, 0, 0);
```

```
    if (SDL_WriteSurfacePixel(surface, 10, 10, 255, 0, 0, 1)) {
```

SDL_Just_a_point.cpp

SDL_Just_a_point

```
25
26
27
28
29 // 2
30 SDL_V
31 windo
32 if (v
33
34
35
36
37 else
38
```

C:\EigeneLokaleDaten\CPP\SDL_Just_a_point\x64\Debug\SDL_Just_a_point.exe

```
Just a point
Window initialization failed
Window created successfully
Pixel written successfully
```

SDL3 Start



Pixel mit SDL



```
#include <iostream>
#include <SDL3/SDL.h>
using namespace std;
```

```
int main()
{
```

```
    cout << "Just a point\n";
```

```
    // 1) Init
```

```
    if (SDL_Init(SDL_INIT_VIDEO)) {
        cout << "Window initialization failed\n";
    }
```

```
    else {
        cout << SDL_GetError();
        SDL_Quit();
        return -1;
    }
```

```
    // 2) create window
```

```
    SDL_Window* window = SDL_CreateWindow(
        "Just a point",
        SDL_WINDOWPOS_CENTERED,
        SDL_WINDOWPOS_CENTERED,
        640, 480,
        SDL_WINDOW_VULKAN);
    if (window == nullptr) {
        cout << "ERROR creating window\n";
        SDL_Quit();
        return -1;
    }
```

```
    else cout << "Window created successfully\n";
```

```
    // 3 Write to surface directly
```

```
    // You may not combine this with 3D or the rendering API on this window.
```

```
    SDL_Surface* surface = SDL_GetWindowSurface(window);
```

```
    SDL_ClearSurface(surface, 0, 0, 0, 0);
```

```
    if (SDL_WriteSurfacePixel(surface, 10, 10, 255, 0, 0, 1)) {
```

SDL_Just_a_point.cpp

SDL_Just_a_point

25

26

27

28

29

30

31

32

33

34

35

36

37

38

Just a point

Window initialization failed

Window created successfully

Pixel written successfully

```
    // 2
```

```
    SDL_V
```

```
    windo
```

```
    if (v
```

```
    }
```

```
    }
```

```
    }
```

```
    }
```

```
    }
```

```
    }
```

```
    }
```

```
    }
```

```
    }
```

```
    // 2
```

```
    SDL_V
```

```
    windo
```

```
    if (v
```

```
    }
```

```
    }
```

```
    }
```

```
    }
```

```
    }
```

```
    }
```

```
    }
```

```
    }
```

```
    }
```

C:\EigeneLokaleDaten\CPP\SDL_Just_a_point\x64\Debug\SDL_Just_a_point.exe

SDL3 start

K
A

SDL zu VS Projekt hinzufügen

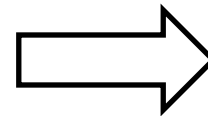


```
#include <iostream>
#include <SDL3/SDL.h>
using namespace std;
```

Download SDL3-devel-3.2.16-VC.zip from
<https://github.com/libSDL-org/SDL/releases/tag/release-3.2.16>

To use SDL3 package, edit your project properties:

1. Add the include directory to
"VC++ Directories" -> "Include Directories"
e.g. D:\Programme\SDL3-3.2.4\include
2. Add the lib / _arch_ directory to
"VC++ Directories" -> "Library Directories"
e.g. D:\Programme\SDL3-3.2.4\lib\x64
3. Add SDL3.lib to Linker-> Input -> "Additional Dependencies"
4. Copy from lib / _arch_ / file SDL3.dll to your project directory.



Ausführliche Anleitung in ILIAS



SDL: Initialisieren



```
#include <iostream>
#include <SDL3/SDL.h>
using namespace std;

int main()
{
    // 1) Init
    if (SDL_Init(SDL_INIT_VIDEO)) {
        cout << "SDL initialization is successfull.\n";
    }
    else {
        cout << SDL_GetError();
        SDL_Quit();
        return -1;
    }
}
```



SDL: Initialisierungs Flags



```
#include <iostream>
#include <SDL3/SDL.h>
using namespace std;

int main()
{
    // 1) Init
    if (SDL_Init(SDL_INIT_VIDEO)) {
        cout << "SDL initialization is successfull.\n";
    }
    else {
        cout << SDL_GetError();
        SDL_Quit();
        return -1;
    }
}
```

Syntax:

```
bool SDL_Init(SDL_InitFlags flags);
```

Wichtigsten SDL_InitFlags:

SDL_INIT_VIDEO
SDL_INIT_AUDIO
SDL_INIT_EVENT

kombinierbar: z.B.

```
SDL_Init(SDL_INIT_VIDEO | SDL_INIT_AUDIO | SDL_INIT_EVENTS)
```

https://wiki.libsdl.org/SDL3/SDL_Init

<https://wiki.libsdl.org/SDL3/CategoryInit>



SDL: Initialisierungs Flags



```
#include <iostream>
#include <SDL3/SDL.h>
using namespace std;

int main()
{
    // 1) Init
    if (SDL_Init(SDL_INIT_VIDEO)) {
        cout << "SDL initialization is successfull.\n";
    }
    else {
        cout << SDL_GetError();
        SDL_Quit();
        return -1;
    }
}
```

Alle SDL_InitFlags:

- **SDL_INIT_AUDIO:**
audio subsystem; automatically initializes the events subsystem
- **SDL_INIT_VIDEO:**
video subsystem; automatically initializes the events subsystem, should be initialized on the main thread.
- **SDL_INIT_JOYSTICK:**
joystick subsystem; automatically initializes the events subsystem
- **SDL_INIT_HAPTIC:**
haptic (force feedback) subsystem
- **SDL_INIT_GAMEPAD:**
gamepad subsystem; automatically initializes the joystick subsystem
- **SDL_INIT_EVENTS:**
events subsystem
- **SDL_INIT_SENSOR:**
sensor subsystem; automatically initializes the events subsystem
- **SDL_INIT_CAMERA:**
camera subsystem; automatically initializes the events subsystem

https://wiki.libsdl.org/SDL3/SDL_Init

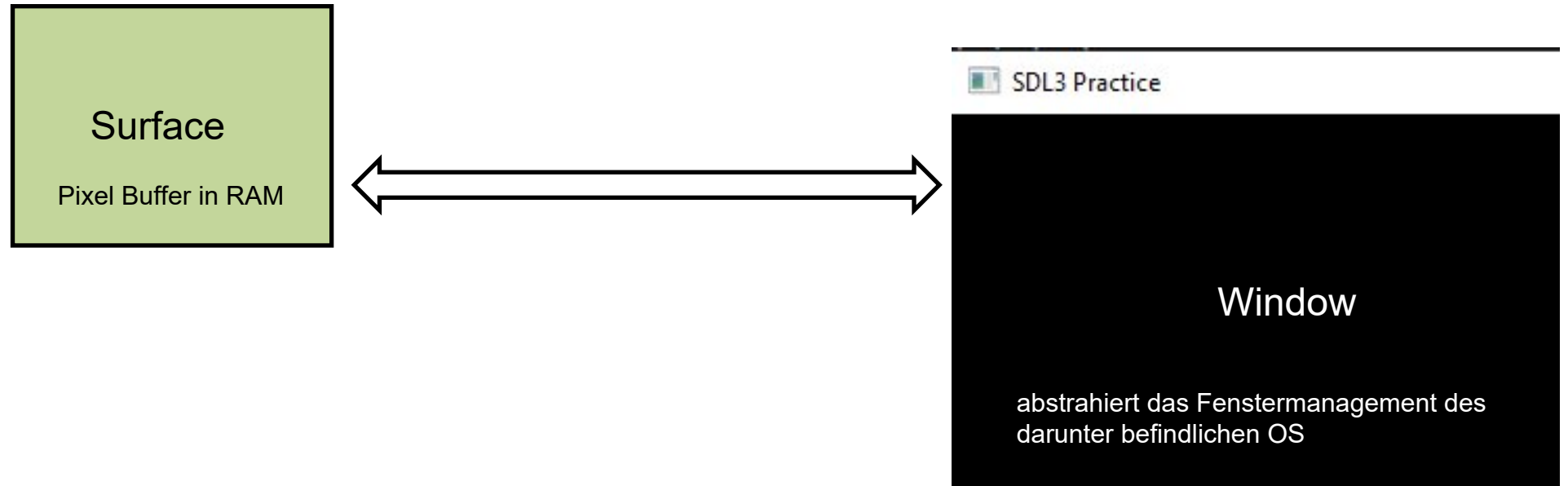
<https://wiki.libsdl.org/SDL3/CategoryInit>



SDL: Window & Surface



Langsam:



SDL: Fenster erzeugen



```
#include <iostream>
#include <SDL3/SDL.h>
using namespace std;

int main()
{
    // 1) Init
    if (SDL_Init(SDL_INIT_VIDEO)) {
        cout << "SDL initialization is successfull.\n";
    }
    else {
        cout << SDL_GetError();
        SDL_Quit();
        return -1;
    }

    // 2) create window
    SDL_Window* window = nullptr;
    window = SDL_CreateWindow("SDL3 Start", 1280, 720,
        SDL_WINDOW_VULKAN | SDL_WINDOW_HIGH_PIXEL_DENSITY);
    if (window == nullptr) {
        cout << "ERROR creating window\n";
        SDL_Quit();
        return -1;
    }
    else cout << "Window created sucessfully.\n";
}
```

Syntax:

```
SDL_Window * SDL_CreateWindow(const char *title,
                               int w, int h,
                               SDL_WindowFlags flags);
```

Ein paar SDL_WindowFlags:

- **SDL_WINDOW_FULLSCREEN:**
fullscreen window at desktop resolution
- **SDL_WINDOW_HIDDEN:**
window is not visible
- **SDL_WINDOW_MAXIMIZED:**
window is maximized
- **SDL_WINDOW_HIGH_PIXEL_DENSITY:**
window uses high pixel density back buffer if possible
- **SDL_WINDOW_VULKAN:**
window usable with a Vulkan (*~Next Generation OpenGL*) instance

https://wiki.libsdl.org/SDL3/SDL_CreateWindow



SDL: Fenster erzeugen



```
#include <iostream>
#include <SDL3/SDL.h>
using namespace std;

int main()
{
    // 1) Init
    if (SDL_Init(SDL_INIT_VIDEO)) {
        cout << "SDL initialization is successfull.\n";
    }
    else {
        cout << SDL_GetError();
        SDL_Quit();
        return -1;
    }

    // 2) create window
    SDL_Window* window = nullptr;
    window = SDL_CreateWindow("SDL3 Start", 1280, 720,
        SDL_WINDOW_VULKAN | SDL_WINDOW_HIGH_PIXEL_DENSITY);
    if (window == nullptr) {
        cout << "ERROR creating window\n";
        SDL_Quit();
        return -1;
    }
    else cout << "Window created sucessfully.\n";
}
```

PC-Pool ↗

Syntax:

```
SDL_Window * SDL_CreateWindow(const char *title,
                               int w, int h,
                               SDL_WindowFlags flags);
```

Ein paar SDL_WindowFlags:

- **SDL_WINDOW_FULLSCREEN:**
fullscreen window at desktop resolution
- **SDL_WINDOW_HIDDEN:**
window is not visible
- **SDL_WINDOW_MAXIMIZED:**
window is maximized
- **SDL_WINDOW_HIGH_PIXEL_DENSITY:**
window uses high pixel density back buffer if possible
- **SDL_WINDOW_VULKAN:**
window usable with a Vulkan (~Next Generation OpenGL) instance

https://wiki.libsdl.org/SDL3/SDL_CreateWindow



SDL: Fenster erzeugen



```
#include <iostream>
#include <SDL3/SDL.h>
using namespace std;

int main()
{
    // 1) Init
    if (SDL_Init(SDL_INIT_VIDEO)) {
        cout << "SDL initialization is successfull.\n";
    }
    else {
        cout << SDL_GetError();
        SDL_Quit();
        return -1;
    }

    // 2) create window
    SDL_Window* window = nullptr;
    window = SDL_CreateWindow("SDL3 Start", 1280, 720,
        SDL_WINDOW_OPENGL | SDL_WINDOW_HIGH_PIXEL_DENSITY);
    if (window == nullptr) {
        cout << "ERROR creating window\n";
        SDL_Quit();
        return -1;
    }
    else cout << "Window created sucessfully.\n";
}
```

Syntax:

```
SDL_Window * SDL_CreateWindow(const char *title,
                               int w, int h,
                               SDL_WindowFlags flags);
```

Ein paar SDL_WindowFlags:

- **SDL_WINDOW_FULLSCREEN:**
fullscreen window at desktop resolution
- **SDL_WINDOW_HIDDEN:**
window is not visible
- **SDL_WINDOW_MAXIMIZED:**
window is maximized
- **SDL_WINDOW_HIGH_PIXEL_DENSITY:**
window uses high pixel density back buffer if possible
- **SDL_WINDOW_VULKAN:**
window usable with a Vulkan (*~Next Generation OpenGL*) instance
- **SDL_WINDOW_OPENGL:**
window usable with an OpenGL context

https://wiki.libsdl.org/SDL3/SDL_CreateWindow





Ein Surface (Oberfläche)

- befindet sich im Hauptspeicher des Computers
- ist entsprechend langsam

```
// 3 Write to surface directly  
// You may not combine this with 3D or rendering on this window.  
SDL_Surface* surface = SDL_GetWindowSurface(window);  
SDL_ClearSurface(surface, 0, 0, 0, 0);  
  
// 4) Clean up & close  
SDL_DestroySurface(surface);  
SDL_DestroyWindow(window);  
SDL_Quit();  
}
```



```
// 3 Write to surface directly
// You may not combine this with 3D or the rendering API on this window.
SDL_Surface* surface = SDL_GetWindowSurface(window);
SDL_ClearSurface(surface, 0, 0, 0, 0);
if (SDL_WriteSurfacePixel(surface, 10, 10, 255, 0, 0, 1)) {
    cout << "Pixel written successfully\n";
}
else {
    cout << "ERROR: Pixel\n";
}
SDL_UpdateWindowSurface(window);

SDL_Delay(5000); // 5 Sekunden warten

// 4) Clean up & close
SDL_DestroyWindow(window);
SDL_Quit();
}
```



Benutzen des **SDL Surface des aktuellen Fensters**:

Syntax:

```
SDL_Surface * SDL_GetWindowSurface(SDL_Window *window);
```

Punkt setzen:

Syntax:

```
bool SDL_WriteSurfacePixel(SDL_Surface *surface,
                           int x, int y,
                           Uint8 r, Uint8 g, Uint8 b, Uint8 a);
```

https://wiki.libsdl.org/SDL3/SDL_WriteSurfacePixel

SDL: Punkte auf Surface



Benutzen des **SDL Surface des aktuellen Fensters**:

Syntax:

```
SDL_Surface * SDL_GetWindowSurface(SDL_Window *window);
```

```
// 3 Write to surface directly
// You may not combine this with 3D or the rendering API on this window.
SDL_Surface* surface = SDL_GetWindowSurface(window);
SDL_ClearSurface(surface, 0, 0, 0, 0);
if (SDL_WriteSurfacePixel(surface, 10, 10, 255, 0, 0, 1)) {
    cout << "Pixel written successfully\n";
}
else {
    cout << "ERROR: Pixel\n";
}
SDL_UpdateWindowSurface(window);

SDL_Delay(5000); // 5 Sekunden warten

// 4) Clean up & close
SDL_DestroyWindow(window);
SDL_Quit();
}
```

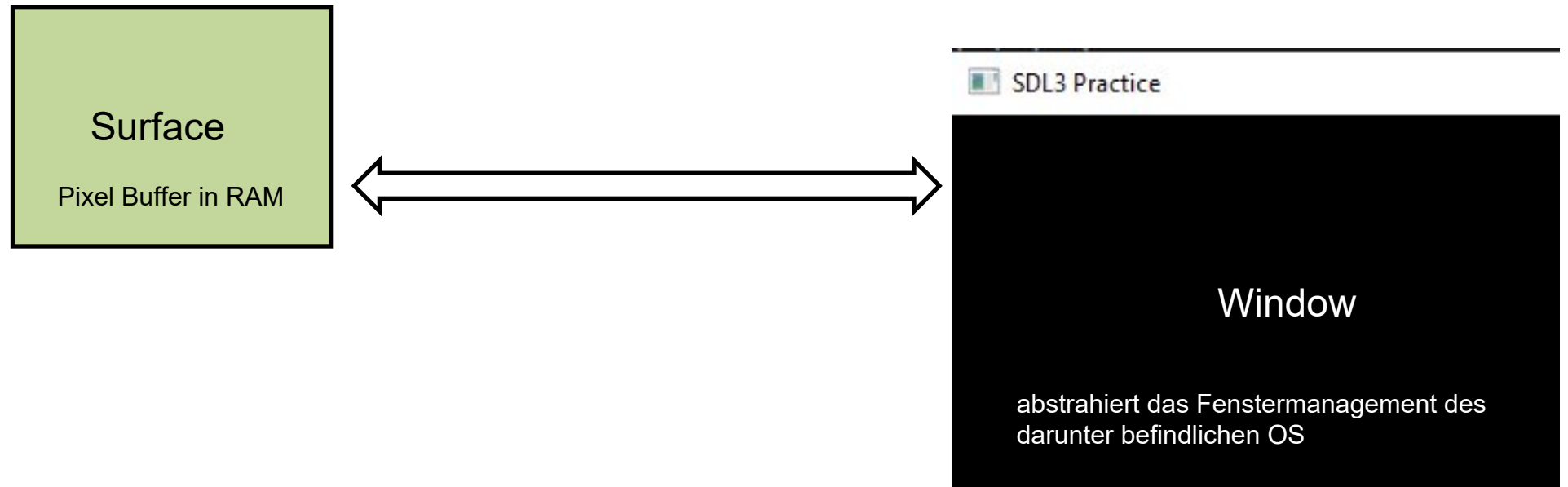
Da sich der Surface im Hauptspeicher des Computers befindet ist dieser Ansatz sehr langsam! ⇒ Verwenden von Grafikspeicher



SDL: Window & Surface



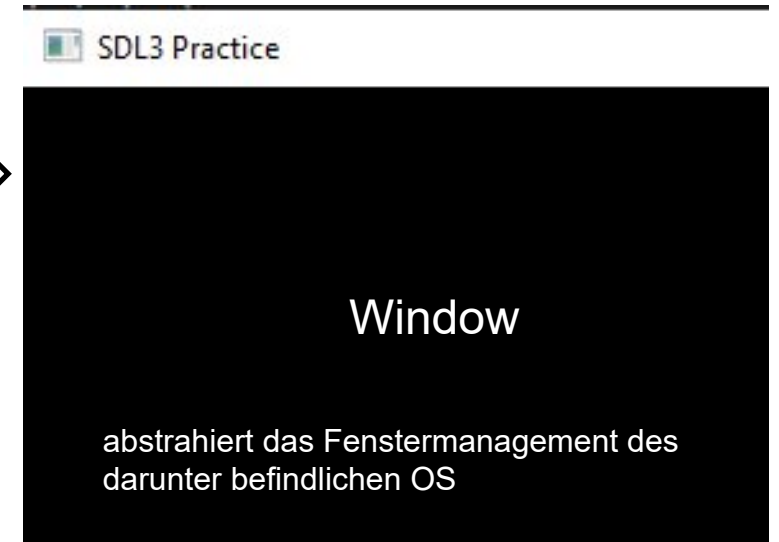
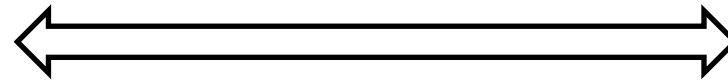
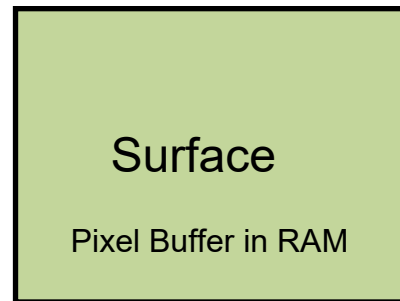
Langsam:



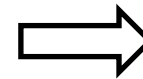
SDL: Window, Surface & Renderer



Langsam:



Schnell:



SDL: Renderer



```
// 1) Init [..]
// 2) create Window [..]
// 3) Renderer
SDL_Renderer* renderer = SDL_CreateRenderer(window, nullptr);
if (renderer == nullptr) {
    cout << "ERROR: could not open renderer\n";
    SDL_DestroyWindow(window);
    SDL_Quit();
    return -1;
}

// 4) Clean up & close
SDL_DestroyRenderer(renderer);
SDL_DestroyWindow(window);
SDL_Quit();

}
```

Renderer:

Ein abstraktes Objekt, welche das 2D Zeichnen auf dem Window übernimmt.

<https://wiki.libsdl.org/SDL3/CategoryRender>

SDL: Renderer



```
// 1) Init [..]
// 2) create Window [..]
// 3) Renderer
SDL_Renderer* renderer = SDL_CreateRenderer(window, nullptr);
if (renderer == nullptr) {
    cout << "ERROR: could not open renderer\n";
    SDL_DestroyWindow(window);
    SDL_Quit();
    return -1;
}

// 4) Clean up & close
SDL_DestroyRenderer(renderer);
SDL_DestroyWindow(window);
SDL_Quit();

}
```

Renderer:

Ein abstraktes Objekt, welche das 2D Zeichnen auf dem Window übernimmt.

<https://wiki.libsdl.org/SDL3/CategoryRender>

Starten

SDL: Renderer (Punkt)



```
// 1) Init [..]
// 2) create Window [..]
// 3) Renderer
SDL_Renderer* renderer = SDL_CreateRenderer(window, nullptr);
if (renderer == nullptr) {
    cout << "ERROR: could not open renderer\n";
    SDL_DestroyWindow(window);
    SDL_Quit(); return -1; }

SDL_SetRenderDrawColor(renderer, 0, 0, 0, 0); // color for cls
SDL_RenderClear(renderer); // cls

SDL_SetRenderDrawColor(renderer, 255, 0, 0, 255); // color pxl
SDL_RenderPoint(renderer, 10, 10);

SDL_RenderPresent(renderer); // update (double buffering)
SDL_Delay(1000); // 1sec warten

// 4) Clean up & close
SDL_DestroyRenderer(renderer);
SDL_DestroyWindow(window);
SDL_Quit();
}
```

Renderer:

Ein abstraktes Objekt, welche das 2D Zeichnen auf dem Window übernimmt.

<https://wiki.libsdl.org/SDL3/CategoryRender>

} cls
} Pixel setzen

SDL: Renderer (Punkt)



```
// 1) Init [..]
// 2) create Window [..]
// 3) Renderer
SDL_Renderer* renderer = SDL_CreateRenderer(window, nullptr);
if (renderer == nullptr) {
    cout << "ERROR: could not open renderer\n";
    SDL_DestroyWindow(window);
    SDL_Quit(); return -1; }

SDL_SetRenderDrawColor(renderer, 0, 0, 0, 0); // color for cls
SDL_RenderClear(renderer); // cls

SDL_SetRenderDrawColor(renderer, 255, 0, 0, 255); // color pxl
SDL_RenderPoint(renderer, 10, 10);

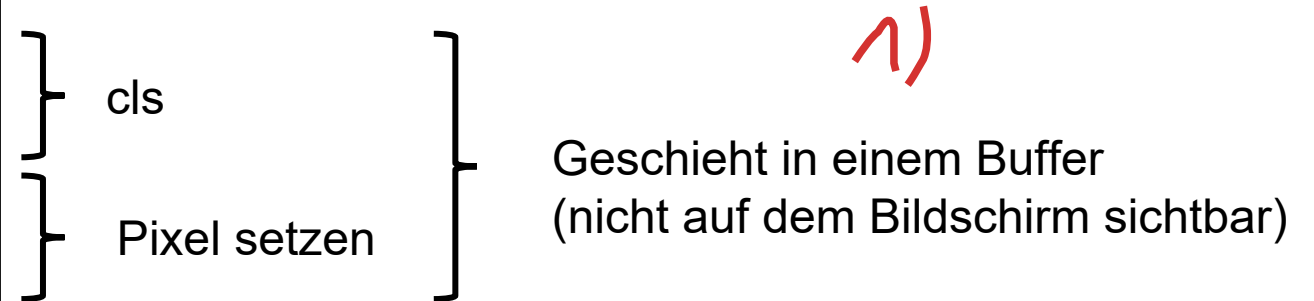
SDL_RenderPresent(renderer); // update (double buffering)
SDL_Delay(1000); // 1sec warten

// 4) Clean up & close
SDL_DestroyRenderer(renderer);
SDL_DestroyWindow(window);
SDL_Quit();
}
```

Renderer:

Ein abstraktes Objekt, welche das 2D Zeichnen auf dem Window übernimmt.

<https://wiki.libsdl.org/SDL3/CategoryRender>



SDL: Renderer (Punkt)



```
// 1) Init [..]
// 2) create Window [..]
// 3) Renderer
SDL_Renderer* renderer = SDL_CreateRenderer(window, nullptr);
if (renderer == nullptr) {
    cout << "ERROR: could not open renderer\n";
    SDL_DestroyWindow(window);
    SDL_Quit(); return -1; }

SDL_SetRenderDrawColor(renderer, 0, 0, 0, 0); // color for cls
SDL_RenderClear(renderer); // cls

SDL_SetRenderDrawColor(renderer, 255, 0, 0, 255); // color pxl
SDL_RenderPoint(renderer, 10, 10);

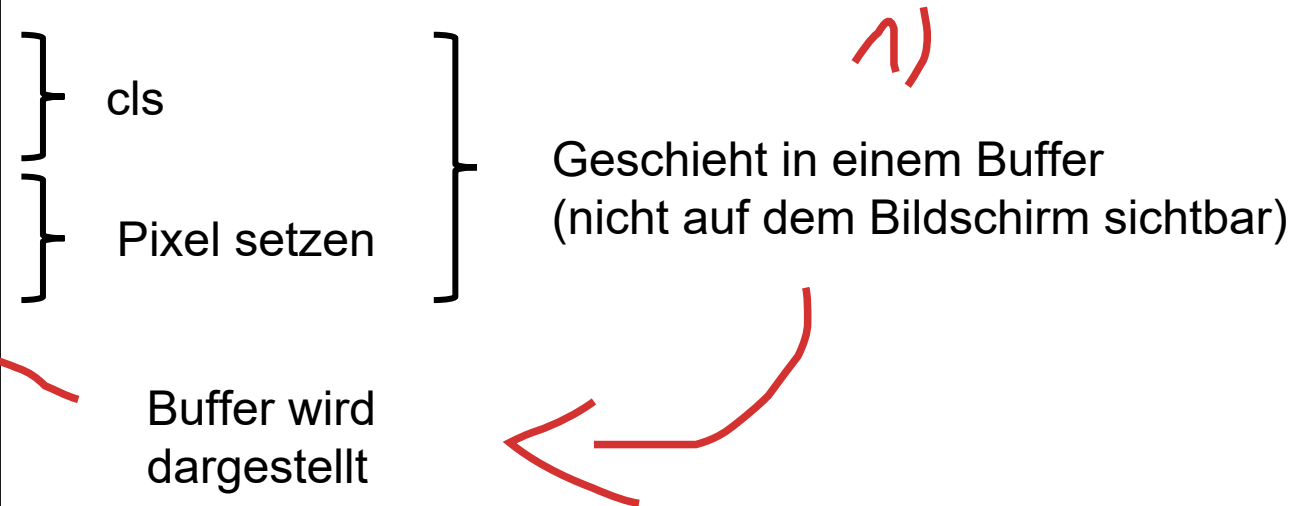
SDL_RenderPresent(renderer); // update (double buffering)
SDL_Delay(1000); // 1sec warten

// 4) Clean up & close
SDL_DestroyRenderer(renderer);
SDL_DestroyWindow(window);
SDL_Quit();
}
```

Renderer:

Ein abstraktes Objekt, welche das 2D Zeichnen auf dem Window übernimmt.

<https://wiki.libsdl.org/SDL3/CategoryRender>



SDL: Double Buffering



```
// 1) Init [..]
// 2) create Window [..]
// 3) Renderer
SDL_Renderer* renderer = SDL_CreateRenderer(window, nullptr);
if (renderer == nullptr) {
    cout << "ERROR: could not open renderer\n";
    SDL_DestroyWindow(window);
    SDL_Quit(); return -1; }

SDL_SetRenderDrawColor(renderer, 0, 0, 0, 0); // color for cls
SDL_RenderClear(renderer); // cls

SDL_SetRenderDrawColor(renderer, 255, 0, 0, 255); // color pxl
SDL_RenderPoint(renderer, 10, 10);

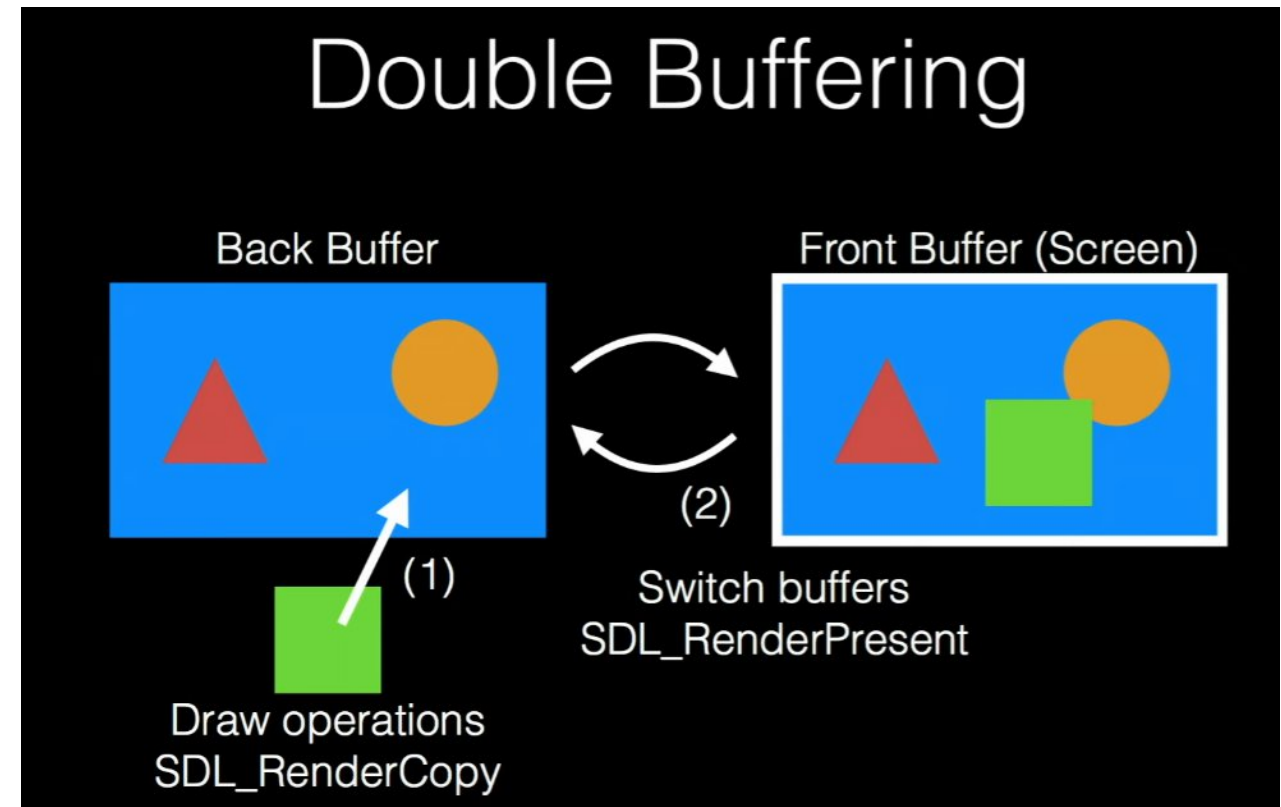
SDL_RenderPresent(renderer); // update (double buffering)
SDL_Delay(1000); // 1sec warten

// 4) Clean up & close
SDL_DestroyRenderer(renderer);
SDL_DestroyWindow(window);
SDL_Quit();
}
```

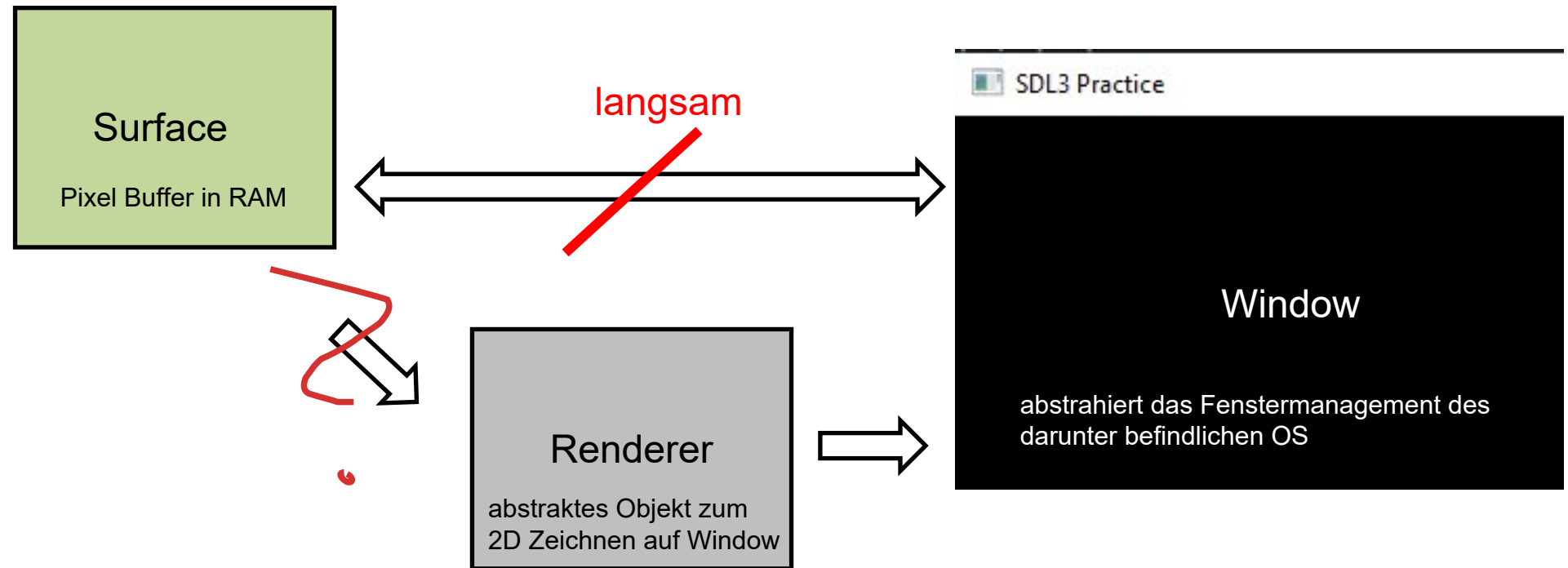
Renderer:

Ein abstraktes Objekt, welche das 2D Zeichnen auf dem Window übernimmt.

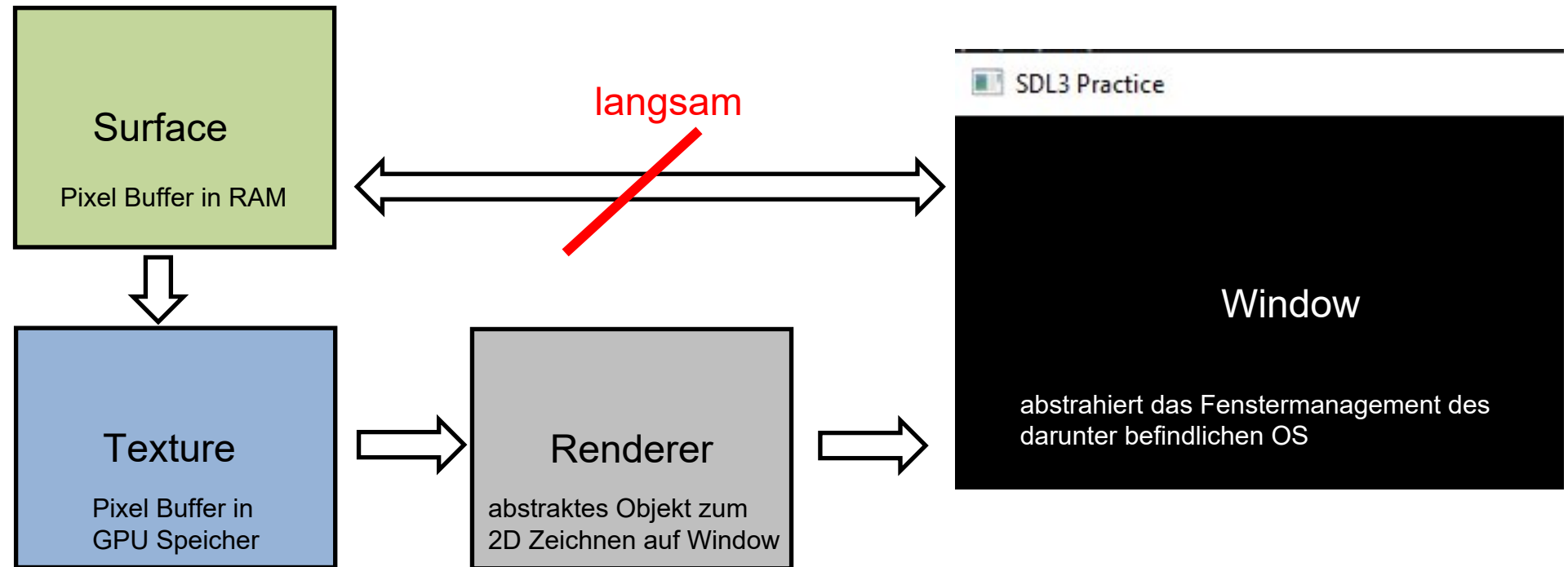
<https://wiki.libsdl.org/SDL3/CategoryRender>



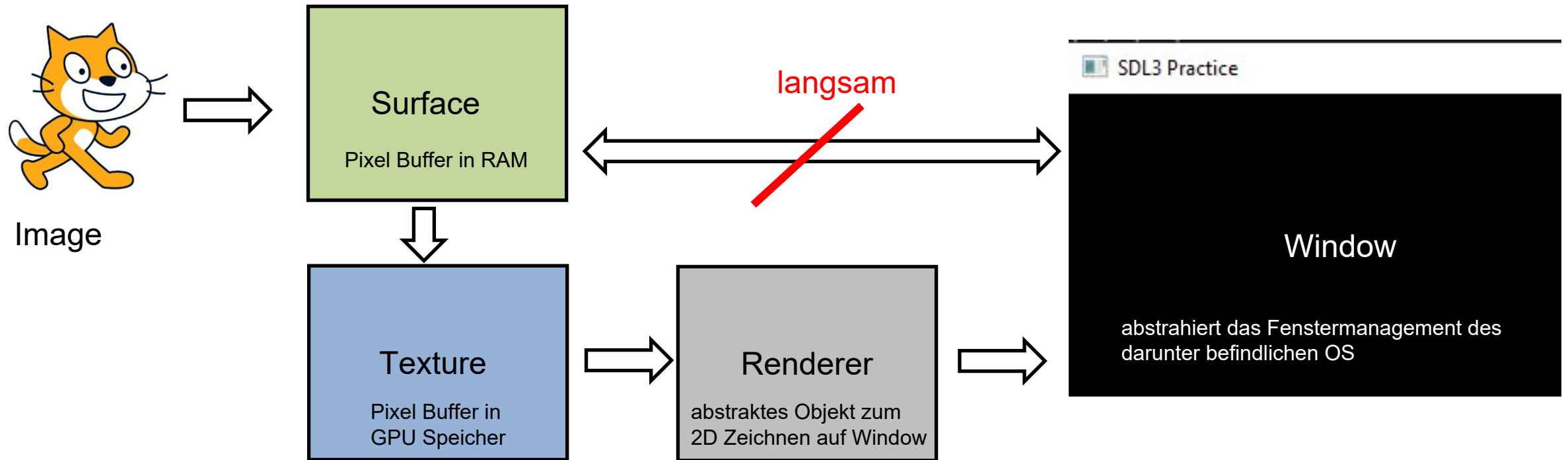
SDL: Window, Surface, Texture & Renderer



SDL: Window, Surface, Texture & Renderer



SDL: Window, Surface, Texture & Renderer



SDL: Texture



```
// 1) Init [..]
// 2) create Window [..]
// 3) Renderer [..]
// 4) Surface
SDL_Surface* surface = SDL_LoadBMP("animal.bmp");
if (surface == nullptr) {
    cout << "ERROR: could not open BMP file \n";
}
// 5) Texture
SDL_Texture* texture = SDL_CreateTextureFromSurface(
    renderer, surface);
if (texture == nullptr) {
    cout << "ERROR: could not generate texture from surface\n";
    SDL_DestroySurface(surface);
    SDL_DestroyRenderer(renderer);
    SDL_DestroyWindow(window);
    SDL_Quit();
    return -1;
}

// 4) Clean up & close
SDL_DestroySurface(surface);
SDL_DestroyTexture(texture);
SDL_DestroyRenderer(renderer);
SDL_DestroyWindow(window);
SDL_Quit();
}
```

Eine Textur (engl. texture)

- befindet sich im Speicher der Grafikkarte
- ist entsprechend schnell!



SDL: Texture



```
// 1) Init [..]
// 2) create Window [..]
// 3) Renderer [..]
// 4) Surface
SDL_Surface* surface = SDL_LoadBMP("animal.bmp");
if (surface == nullptr) {
    cout << "ERROR: could not open BMP file \n";
}
// 5) Texture
SDL_Texture* texture = SDL_CreateTextureFromSurface(
    renderer, surface);
if (texture == nullptr) {
    cout << "ERROR: could not generate texture from surface\n";
    SDL_DestroySurface(surface);
    SDL_DestroyRenderer(renderer);
    SDL_DestroyWindow(window);
    SDL_Quit();
    return -1;
}

// 4) Clean up & close
SDL_DestroySurface(surface);
SDL_DestroyTexture(texture);
SDL_DestroyRenderer(renderer);
SDL_DestroyWindow(window);
SDL_Quit();
}
```

Eine Textur (engl. texture)

- befindet sich im Speicher der Grafikkarte
- ist entsprechend schnell!

Bildinformationen in den
RAM (Surface) laden



SDL: Texture



```
// 1) Init [..]
// 2) create Window [..]
// 3) Renderer [..]
// 4) Surface
SDL_Surface* surface = SDL_LoadBMP("animal.bmp");
if (surface == nullptr) {
    cout << "ERROR: could not open BMP file \n";
}
// 5) Texture
SDL_Texture* texture = SDL_CreateTextureFromSurface(
    renderer, surface);
if (texture == nullptr) {
    cout << "ERROR: could not generate texture from surface\n";
    SDL_DestroySurface(surface);
    SDL_DestroyRenderer(renderer);
    SDL_DestroyWindow(window);
    SDL_Quit();
    return -1;
}

// 4) Clean up & close
SDL_DestroySurface(surface);
SDL_DestroyTexture(texture);
SDL_DestroyRenderer(renderer);
SDL_DestroyWindow(window);
SDL_Quit();
}
```

Eine Textur (engl. texture)

- befindet sich im Speicher der Grafikkarte
- ist entsprechend schnell!



Textur aus Surface erzeugen:
Informationen in den
Grafikkartenspeicher kopieren.

SDL: Texture



```
// 1) Init [..]
// 2) create Window [..]
// 3) Renderer [..]
// 4) Surface
SDL_Surface* surface = SDL_LoadBMP("animal.bmp");
if (surface == nullptr) {
    cout << "ERROR: could not open BMP file \n"; }
// 5) Texture
SDL_Texture* texture = SDL_CreateTextureFromSurface(
    renderer, surface);

if (texture == nullptr) {
    cout << "ERROR: could not generate texture from surface\n"; }
SDL_DestroySurface(surface); // free memory

SDL_RenderClear(renderer); // cls

SDL_FRect dest; dest.x = 100; dest.y = 100;
SDL_RenderTexture(renderer, texture, nullptr, &dest);

SDL_RenderPresent(renderer); // update (double buffering)
SDL_Delay(1000); // 1sec warten

// 4) Clean up & close
SDL_DestroyRenderer(renderer);
SDL_DestroyWindow(window);
SDL_Quit();
}
```

Eine Textur (engl. texture)

- befindet sich im Speicher der Grafikkarte
- ist entsprechend schnell!



An xy-Position (100,100) darstellen



SDL: Texture



```
// 1) Init [..]
// 2) create Window [..]
// 3) Renderer [..]
// 4) Surface
SDL_Surface* surface = SDL_LoadBMP("animal.bmp");
if (surface == nullptr) {
    cout << "ERROR: could not open BMP file \n"; }
// 5) Texture
SDL_Texture* texture = SDL_CreateTextureFromSurface(
    renderer, surface);

if (texture == nullptr) {
    cout << "ERROR: could not generate texture from surface\n";
    SDL_DestroySurface(surface); // free memory

    SDL_RenderClear(renderer); // cls

    SDL_FRect dest; dest.x = 100; dest.y = 100;
    dest.w = 200; dest.h = 180;
    SDL_RenderTexture(renderer, texture, nullptr, &dest);

    SDL_RenderPresent(renderer); // update (double buffering)
    SDL_Delay(1000); // 1sec warten

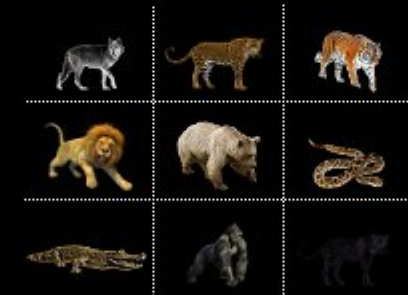
    // 4) Clean up & close
    SDL_DestroyRenderer(renderer);
    SDL_DestroyWindow(window);
    SDL_Quit();
}
```

Fine Textur (engl. texture)

C:\EigeneLokaleDaten\CPP\SDL3_getting_started\x64\Debug\SDL3_getting_started.exe

```
Window initialization is successfull.
loaded audio file..
Audio Stream started.....
Window created successfully.
last SDL error code:
Loaded Texture with w: 194 and h: 144
```

SDL3 Practice



SDL: Events



```
bool down = false;
[.]

while (!done){
    // process events..
    SDL_Event event;
    while (SDL_PollEvent(&event)) {
        switch (event.type) {
            case SDL_EVENT_QUIT: // Quit triggered...
                done = true;
                break;
            case SDL_EVENT_KEY_DOWN: // Taste gedrückt
                switch (event.key.scancode) {
                    case SDL_SCANCODE_DOWN:
                    case SDL_SCANCODE_S:
                        down = true;
                        break;
                }
            [.]

            case SDL_EVENT_KEY_UP: // Taste losgelassen
                switch (event.key.scancode) {
                    case SDL_SCANCODE_DOWN:
                    case SDL_SCANCODE_S:
                        down = false;
                        break;
                }
            [.]
        }
    }
}
```

SDL speichert alle Events (Ereignisse) in einer Queue (Warteschlange). Mit `SDL_PollEvent()` werden die Events von der Queue abgerufen.

Syntax:

```
bool SDL_PollEvent(SDL_Event *event);
```

Somit lassen sich Tastatureingaben, Mausinteraktion, etc. abfragen.

<https://wiki.libsdl.org/SDL3/CategoryEvents>



SDL: Referenzen

Main Web Page SDL3:

<https://wiki.libsdl.org/SDL3/FrontPage>

<https://wiki.libsdl.org/SDL3/APIByCategory>

Code Examples SDL3:

<https://examples.libsdl.org/SDL3/>

Lectures:

Writing 2D Games in C using SDL by Thomas Lively

<https://youtu.be/yFLa3ln16w0?t=1061>

Creating a Game Loop with C & SDL (Tutorial)

<https://youtu.be/XfZ6WrV5Z7Y?t=524>

How to use ImGui in SDL?

<https://youtu.be/Yg440qAekww>

Kleines Tutorial (SDL1)

<https://www.spieleprogrammierer.de/wiki/SDL-Tutorial>

Grafik: Referenzen



Tonc, a guide to Game Boy Advance programming

<https://gbadev.net/tonc/mode7.html>

What is Mode 7? Let's code it!

<https://youtu.be/D0MPYZYe40E>

Programming Pseudo 3D Planes aka MODE7 (C++)

<https://youtu.be/ybLZyY655iY>

Mode7 demo, topdown and first person view

<https://youtu.be/6OZyaIMDFmA>

<https://github.com/olivier1980/mode7new>

SDL: Audio



```
// AUDIO STUFF!!!
Uint8* wav_data = nullptr;
Uint32 wav_data_length = 0;
SDL_AudioSpec spec;
if (SDL_LoadWAV("APT.wav", &spec,
               &wav_data, &wav_data_length)) {
    cout << "loaded audio file..\n";
}
else {
    cout << "ERROR: failed to load audio...\n";
}

SDL_AudioStream* stream;
stream =
SDL_OpenAudioDeviceStream(SDL_AUDIO_DEVICE_DEFAULT_PL
AYBACK, &spec, nullptr, nullptr);
if (stream == nullptr) {
    cout << "could not load audio stream\n";
    return -1;
}
else {
    cout << "Audio Stream started.....\n";
}
SDL_ResumeAudioStreamDevice(stream);
if (SDL_GetAudioStreamQueued(stream) < (int)wav_data_length) {
    SDL_PutAudioStreamData(stream, wav_data,
wav_data_length);
}
// END AUDIO STUFF
```

Folie wird später (d.h. irgendwann) ergänzt.....
Und erklärt.....



3D → Voxel



Folie wird später (d.h. irgendwann) ergänzt.....
Und erklärt.....

- Zuerst war der Punkt
- Feuerdemo
- 2D Grafik
 - Sprites
 - Bitmaps laden & darstellen
- 2.5D Grafik
 - Mode 7
 - Raycasting
- 3D Grafik
 - Polygonen
 - Voxel
 - Raytracing

C++: Simple DirectMedia Layer (SDL)



„Modulo“ Operator



```
// 724ms
inline int mod1(int a, int b)
{
    const int r = a % b;
    return r < 0 ? r + b : r;
}

// 759ms
inline int mod2(int a, int b)
{
    return (a % b + b) % b;
}
```

„Der Modulo-Operator führt eine Restwert-Division zweier Elemente aus. Dieser Operator ist nur für Integer-Typen zulässig und entspricht der Berechnung des Restwertes einer Division. [..]

Die Modulo-Berechnung ergibt in C grundsätzlich nur mit positiven Werten sinnvolle Ergebnisse. [..] Wenn der erste Operand negativ ist, so wird die Berechnung ausgeführt, als ob der Operand positiv wäre und schlussendlich dem Resultat das Vorzeichen wieder angehängt.“

<https://stackoverflow.com/questions/11630321/why-does-c-output-negative-numbers-when-using-modulo>
<https://manderc.com/operators/modoperator/index.php>



Der Empfänger Bob

1. wählt zufällige große Primzahlen p und q . Daraus berechnet er

$$n = p \cdot q$$

2. berechnet die Eulersche Φ -Funktion $\phi(n)$, welche die Anzahl der Element x in Z_n mit $ggT(n, x) = 1$ bestimmt:

$$\phi(n) = (p - 1)(q - 1)$$

3. wählt zufällig eine große Zahl $e \in Z_n$ mit $ggT(e, \phi(n)) = 1$ und berechnet d , das Inverse zu e modulo $\phi(n)$ ist, d.h.

$$(d \cdot e) \bmod \phi(n) = 1$$

Der öffentliche Schlüssel ist (n, e) und

der private Schlüssel ist $(p, q, \phi(n), d)$. Wobei $(p, q, \phi(n))$ nicht mehr benötigt und vernichtet werden können.

Verschlüsselung: $f_e(m) = m^e \bmod n = c$

Entschlüsselung: $f_d(c) = c^d \bmod n$

Public-Key Eigenschaft:

Wenn man nur die Zahl n kennt (und nicht etwa die Faktoren p und q oder die Zahl $\phi(n) = (p - 1)(q - 1)$), so kann man aus e nicht d berechnen.

Wieso ist RSA sicher?



Der Empfänger Bob

1. wählt zufällige große Primzahlen p und q . Daraus berechnet er

$$n = p \cdot q$$

2. berechnet die Eulersche Φ -Funktion $\phi(n)$, welche die Anzahl der Element x in Z_n mit $ggT(n, x) = 1$ bestimmt:

$$\phi(n) = (p - 1)(q - 1)$$

3. wählt zufällig eine große Zahl $e \in Z_n$ mit $ggT(e, \phi(n)) = 1$ und berechnet d , das Inverse zu e modulo $\phi(n)$ ist, d.h.

$$(d \cdot e) \bmod \phi(n) = 1$$

Der öffentliche Schlüssel ist (n, e) und

der private Schlüssel ist $(p, q, \phi(n), d)$. Wobei $(p, q, \phi(n))$ nicht mehr benötigt und vernichtet werden können.

Verschlüsselung: $f_e(m) = m^e \bmod n = c$

Entschlüsselung: $f_d(c) = c^d \bmod n$

Public-Key Eigenschaft:

Wenn man nur die Zahl n kennt (und nicht etwa die Faktoren p und q oder die Zahl $\phi(x) = (p - 1)(q - 1)$), so kann man aus e nicht d berechnen.



Wieso ist RSA sicher?



Der Empfänger Bob

1. wählt zufällige große Primzahlen p und q . Daraus berechnet er

$$n = p \cdot q$$

2. berechnet die Eulersche Φ -Funktion $\phi(n)$, welche die Anzahl der Element x in Z_n mit $ggT(n, x) = 1$ bestimmt:

$$\phi(n) = (p - 1)(q - 1)$$

3. wählt zufällig eine große Zahl $e \in Z_n$ mit $ggT(e, \phi(n)) = 1$ und berechnet d , das Inverse zu e modulo $\phi(n)$ ist, d.h.

$$(d \cdot e) \bmod \phi(n) = 1$$

Der öffentliche Schlüssel ist (n, e) und

der private Schlüssel ist $(p, q, \phi(n), d)$. Wobei $(p, q, \phi(n))$ nicht mehr benötigt und vernichtet werden können.

Verschlüsselung: $f_e(m) = m^e \bmod n = c$

Entschlüsselung: $f_d(c) = c^d \bmod n$



1. Ohne Kenntnis der Primfaktoren p und q kann $\phi(n)$ nicht berechnet werden!

Public-Key Eigenschaft:

Wenn man nur die Zahl n kennt (und nicht etwa die Faktoren p und q oder die Zahl $\phi(x) = (p - 1)(q - 1)$), so kann man aus e nicht d berechnen.

K
A

Wieso ist RSA sicher?



Der Empfänger Bob

1. wählt zufällige große Primzahlen p und q . Daraus berechnet er

$$n = p \cdot q$$

2. berechnet die Eulersche Φ -Funktion $\phi(n)$, welche die Anzahl der Element x in Z_n mit $ggT(n, x) = 1$ bestimmt:

$$\phi(n) = (p - 1)(q - 1)$$

3. wählt zufällig eine große Zahl $e \in Z_n$ mit $ggT(e, \phi(n)) = 1$ und berechnet d , das Inverse zu e modulo $\phi(n)$ ist, d.h.

$$(d \cdot e) \bmod \phi(n) = 1$$

Der öffentliche Schlüssel ist (n, e) und

der private Schlüssel ist $(p, q, \phi(n), d)$. Wobei $(p, q, \phi(n))$ nicht mehr benötigt und vernichtet werden können.

Verschlüsselung: $f_e(m) = m^e \bmod n = c$

Entschlüsselung: $f_d(c) = c^d \bmod n$

Public-Key Eigenschaft:

Wenn man nur die Zahl n kennt (und nicht etwa die Faktoren p und q oder die Zahl $\phi(x) = (p - 1)(q - 1)$), so kann man aus e nicht d berechnen.



1. Ohne Kenntnis der Primfaktoren p und q kann $\phi(n)$ nicht berechnet werden!
2. Es existiert kein klassischer Algorithmus zur Primfaktorzerlegung

K
A

Wieso ist RSA sicher?



Ein 2048-Bit RSA-Schlüssel hat 617 Dezimalstellen

Der Empfänger Bob

1. wählt zufällige große Primzahlen p und q . Daraus berechnet er

$$n = p \cdot q$$

2. berechnet die Eulersche Φ -Funktion $\phi(n)$, welche die Anzahl der Element x in Z_n mit $ggT(n, x) = 1$ bestimmt:

$$\phi(n) = (p - 1)(q - 1)$$

3. wählt zufällig eine große Zahl $e \in Z_n$ mit $ggT(e, \phi(n)) = 1$ und berechnet d , das Inverse zu e modulo $\phi(n)$ ist, d.h.

$$(d \cdot e) \bmod \phi(n) = 1$$

Der öffentliche Schlüssel ist (n, e) und

der private Schlüssel ist $(p, q, \phi(n), d)$. Wobei $(p, q, \phi(n))$ nicht mehr benötigt und vernichtet werden können.

Verschlüsselung: $f_e(m) = m^e \bmod n = c$

Entschlüsselung: $f_d(c) = c^d \bmod n$

Public-Key Eigenschaft:

Wenn man nur die Zahl n kennt (und nicht etwa die Faktoren p und q oder die Zahl $\phi(x) = (p - 1)(q - 1)$), so kann man aus e nicht d berechnen.



1. Ohne Kenntnis der Primfaktoren p und q kann $\phi(n)$ nicht berechnet werden!
2. Es existiert kein klassischer Algorithmus zur Primfaktorzerlegung

K
A